

A Mess of Memory System Benchmarking, Simulation and Application Profiling

Pouya Esmaili-Dokht^{*†}, Francesco Sgherzi^{*†}, Valéria Soldera Girelli^{*†}, Isaac Boixaderas^{*}, Mariana Carmin^{*}, Alireza Monemi^{*}, Adrià Armejach^{*†}, Estanislao Mercadal^{*}, Germán Llorc^{*}, Petar Radojković^{*}, Miquel Moreto^{*†}, Judit Giménez^{*†}, Xavier Martorell^{*†}, Eduard Ayguadé^{*†}, Jesus Labarta^{*†}, Emanuele Confalonieri[‡], Rishabh Dubey[‡], Jason Adlard[‡]
 Barcelona Supercomputing Center^{*}, Universitat Politècnica de Catalunya[†], Micron Technology[‡]
 {pouya.esmaili, francesco.sgherzi, valeria.soldera, isaac.boixaderas, mariana.carmin, alireza.monemi, adria.armejach}@bsc.es
 {lau.mercadal, german.llibre, petar.radojkovic, miquel.moreto, judit, xavier.martorell, eduard.ayguade, jesus.labarta}@bsc.es
 {econfalo, rdubeya, jadlard}@micron.com

Abstract—The Memory stress (Mess) framework provides a unified view of the memory system benchmarking, simulation and application profiling.

The *Mess benchmark* provides a holistic and detailed memory system characterization. It is based on hundreds of measurements that are represented as a family of bandwidth–latency curves. The benchmark increases the coverage of all the previous tools and leads to new findings in the behavior of the actual and simulated memory systems. We deploy the Mess benchmark to characterize Intel, AMD, IBM, Fujitsu, Amazon and NVIDIA servers with DDR4, DDR5, HBM2 and HBM2E memory. The *Mess memory simulator* uses bandwidth–latency concept for the memory performance simulation. We integrate Mess with widely-used CPUs simulators enabling modeling of all high-end memory technologies. The Mess simulator is fast, easy to integrate and it closely matches the actual system performance. By design, it enables a quick adoption of new memory technologies in hardware simulators. Finally, the *Mess application profiling* positions the application in the bandwidth–latency space of the target memory system. This information can be correlated with other application runtime activities and the source code, leading to a better overall understanding of the application’s behavior.

The current Mess benchmark release covers all major CPU and GPU ISAs, x86, ARM, Power, RISC-V, and NVIDIA’s PTX. We also release as open source the ZSim, gem5 and OpenPiton Metro-MPI integrated with the Mess simulator for DDR4, DDR5, Optane, HBM2, HBM2E and CXL memory expanders. The Mess application profiling is already integrated into a suite of production HPC performance analysis tools.

I. INTRODUCTION

The importance of the main memory in the overall system’s design [1]–[3] drives significant effort for memory system benchmarking, simulation, and memory-related application profiling. Although these three memory performance aspects are inherently interrelated, they are analyzed with distinct and decoupled tools. **Memory benchmarks** typically report the maximum sustainable memory bandwidth [4]–[6] or performance of the bandwidth-limited application kernels [7]. This is sometimes complemented with latency measurements in unloaded memory systems [8]–[10] or for a small number of memory-usage scenarios [11], [12]. **Memory simulators** determine the memory system response time for a given traffic. Simple simulators model memory with a fixed latency, or

calculate its service time based on queueing theory or simplified DDR protocols [13]–[19]. Dedicated cycle-accurate memory simulators consider detailed memory device sequences and timings [20]–[25]. **Application profiling tools** determine whether applications are memory bound based on the memory access latency [26]–[28], the position in the Roofline model [29], [30] or the the memory-related portion of the overall CPI stack [31].

Our study argues that the memory system benchmarking, simulation and application profiling can and should be based on a **unified view of memory system performance**. We provide this view with the **Memory stress (Mess) framework** comprised of the Mess benchmark, analytical memory system simulator and application profiling tool (Figure 1). **Mess benchmark** (Section II) describes the memory system performance with a **family of bandwidth–latency curves**. The benchmark covers the full range of the memory traffic intensity, from the unloaded to fully-saturated memory system. It also considers numerous compositions of read and write operations, plotted with different shades of blue in Figure 1 (middle). The Mess benchmark is designed for holistic and detailed memory system characterization, and it is easily adaptive to different target platforms. The current benchmark release covers all major CPU and GPU ISAs: **x86, ARM, Power, RISC-V**, and NVIDIA’s **Parallel Thread Execution (PTX)** [32].

We deploy the Mess benchmark to characterize **Intel, AMD, IBM, Fujitsu and Amazon servers** as well as **NVIDIA GPUs with DDR4, DDR5, HBM2 and HBM2E memory** (Section III). We report and discuss a wide range of memory system behavior even for the hardware platforms with the same main memory configuration. These differences are especially pronounced in the high-bandwidth areas which have the greatest impact on memory-intensive applications. We also detect and analyze scenarios in which increase of the memory request rate leads to the degradation of the measured bandwidth.

We use the Mess benchmark to evaluate memory system simulation of event-based **ZSim** [15], cycle-accurate **gem5** [16] and RTL simulator **OpenPiton Metro-MPI** [33] (Section IV). We evaluate different internal memory models and widely-used external memory simulators, **DRAMsim3** [22], **Ramu-**

lator [22] and Ramulator 2 [24]. Unfortunately, all tested memory simulators, including well-established and trusted gem5 DDR models and cycle-accurate memory simulators, poorly resemble the actual system performance. The simulators show an unrealistically low load-to-use latency (starting at 4 ns), high memory bandwidth (exceeding $1.8\times$ the maximum theoretical one), and a simulation error of tens of percents for memory-intensive benchmarks, STREAM [5], LMBench [8] and Google multichase [9]. We detect two sources of these errors: the Zsim interface towards the external memory simulators, and the imprecise DRAMsim3 and Ramulator model of the row-buffer utilization. Finally, although it was not the initial design target, we also detected a case in which holistic and detailed Mess benchmarking diagnosed a bug in the coherency protocol generated by the OpenPiton framework.

Apart from the memory system characterization, the Mess bandwidth–latency curves can be also used for the memory performance simulation (Section V). We develop and integrate the **Mess simulator** in the **ZSim**, **gem5**, and **OpenPiton Metro-MPI simulators**, enabling simulation of high-end memory systems based on **DDR4**, **DDR5**, **Optane**, **HBM2**, and **HBM2E** technologies, and **Compute Express Link (CXL)** [34]. The Mess integration is easy, based on the standard interfaces between the CPU and external memory simulators [15], [16], [35]. The Mess simulator closely matches the actual memory systems performance. The simulation error for STREAM, LMBench and Google multichase benchmarks is between 0.4% and 6%, which is significantly better than any other memory models we tested. Mess is also fast. For example, Mess integrated with ZSim introduces only 26% simulation time increment over the fixed-latency memory model, while the speed-up over the Ramulator and DRAMsim3 ranges between $13\times$ and $15\times$. Mess also removes the current lag between the emergence of memory technologies and development of the reliable simulation models. For example, Mess is the first memory simulator that models **CXL memory expanders**, enabling further research on these novel memory devices. The simulation is based on the bandwidth–latency curves obtained from the memory manufacturer’s SystemC hardware model (Section V-C).

Finally, the Mess memory bandwidth–latency curves enhance application profiling and performance analysis (Section VI). **Mess application profiling** determines positions of application execution time segments on the corresponding memory bandwidth–latency curves. The application memory stress can be combined with the overall application timeline analysis and can be linked to the source code. The Mess application profiling is already integrated into a suite of **production HPC performance analysis tools** [36].

The inherent dependency between the used memory bandwidth and the access latency is by no means a new phenomenon, and the community has known about it at least for a couple of decades [37]. The Mess framework extends the previous work in three important aspects. Previous studies typically use a single bandwidth–latency memory curve to illustrate a **general memory system behavior** [12], [27], [37]–[40]. The Mess

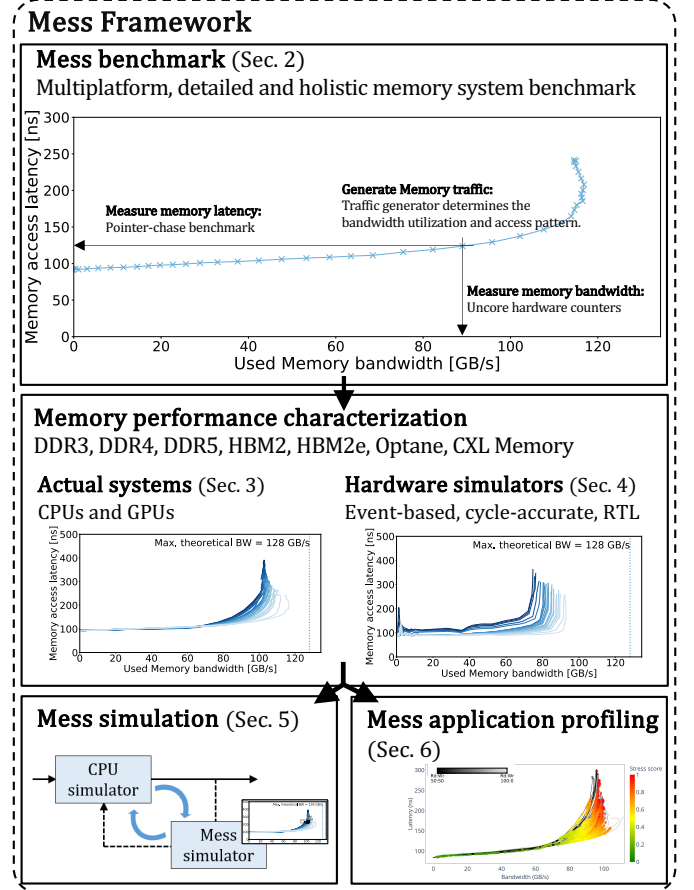


Fig. 1: Mess framework: The Mess benchmark describes the memory system performance with a family of bandwidth–latency curves. We deploy the benchmark to characterize memory systems of actual servers and hardware simulators. The Mess bandwidth–latency curves can be also used for the memory performance simulation and memory-related application profiling.

benchmark is designed for **holistic, detailed and close-to-the-hardware** memory system performance characterization of the **particular system under study**. To reach this objective, the Mess benchmark is developed directly in the assembly, and the experiments are tailored to minimize and mitigate the impact of the system software. It detects and quantifies some aspects of the memory systems behavior not discussed in previous studies, such as the impact of the read and write memory traffic on performance, or discrepancies between different memory systems, both in actual platforms and hardware simulators. Second, the Mess framework **tightly integrates the memory performance characterization into the memory simulation**. The Mess simulator avoids complex memory system simulation and analytically adjusts the rate of memory instructions (provided by the CPU simulator) to the actual memory performance. The accuracy of this simulation approach relies on the input memory performance characterization. This characterization therefore has to be holistic, detailed and specific to the system under

study, closely matching the Mess benchmark design. Third, the framework **closely couples the memory-related profiling of hardware platforms and applications**. Similar to the Mess simulator, the Mess application profiling itself is uncomplicated, and its real value comes from the application analysis in the context of the memory system characteristic. The quality of the memory characterization therefore directly impacts the quality of the overall analysis. For this reason, the analysis has to be performed with detailed Mess-like memory performance description.

The Mess benchmark is released as open source and it is ready to be used in **x86**, **Power**, **ARM**, and **RISC-V** CPUs and **NVIDIA** GPUs [32]. The release also contains all bandwidth–latency measurements shown in the paper, including the **CXL expander curves** provided by the memory manufacturer. We also release as open source the **ZSim**, **gem5** and **OpenPiton Metro-MPI** integrated with the Mess simulator supporting **DDR4**, **DDR5**, **Optane**, **HBM2**, **HBM2E** and **CXL expanders** [41]. Also, the public releases of the **production HPC performance analysis tools** already include the Mess application profiling extension [42]. The released tools are ready to be used by the community for better understanding of the current and exploration of future memory systems.

II. MESS BENCHMARK

This section describes the Mess benchmark use, and analyzes its characterization of actual platforms and hardware simulators.

A. Mess benchmark: Memory bandwidth–latency curves

The Mess memory characterization comprises tens of bandwidth–latency curves, each corresponding to a specific ratio between the read and write memory traffic. The Mess benchmark kernels cover the whole range of memory operations, from 100%-loads to 100%-stores. The 100%-load kernel generates a 100%-read memory traffic, while the 100%-store kernel creates 50%-read/50%-write traffic. This is because the contemporary CPUs deploy the write-allocate cache policy [43]. With this policy, each store instruction first reads data from the main memory to the cache, then modifies it, and finally writes it to the main memory once the cache line is evicted. Each store instruction, therefore, does not correspond to a single memory write, but to one read and one write.¹

The Mess bandwidth–latency curves are illustrated in Figure 1 (middle). The curves with different composition of read and write traffic are plotted with different shades of blue. Each curve is constructed based on tens of measurement points that cover the whole range of memory-traffic intensity. Figure 1 (top) illustrates the construction process for one of the curves. The x -axis of the chart corresponds to memory bandwidth, monitored with hardware counters [44]–[47]. The memory access latency is measured with a pointer-chase

benchmark [48], [49] executed on one CPU core or one GPU Stream Multiprocessor (SM). This determines the y -axis position of the measurement. The pointer-chase is selected because it is simple and portable to different actual hardware platforms and simulators. Optionally, memory latency could be measured with instruction-based sampling approach [50] available in some current architectures [28], [51], [52].

Running pointer-chase alone measures the unloaded memory access latency. To measure the latency of the loaded memory system, concurrently with the pointer-chase, on the remaining CPU cores or GPU SMs, we run a memory traffic generator. The memory access latency is still measured only for the pointer-chase benchmark. The purpose of the traffic generator is to create memory accesses that will collide with the pointer-chase in the loaded memory system. The generator is designed to create memory traffic with configurable memory bandwidth utilization and read/write ratio. Each CPU core and GPU SM traverses two separate arrays, one with load and one with store operations. Therefore, the overall memory traffic is complex, determined by a sequential accesses within each array, but also by the interleaving between memory request from distinct arrays. The Mess benchmark covers a large range of row-buffer hit/empty/miss rates, e.g. between 35/43/22% and 84/13/3% in state-of-the-art Intel architectures (Section IV-D).

Both, pointer-chase and traffic generator are implemented in assembly to minimize any compiler intervention. The detailed implementations of pointer-chase and traffic generator is presented in Appendix A. To minimize the latency penalties introduced by the TLB misses and the page walk, the Mess data-structures are allocated in huge memory pages. Additionally, at runtime, these overheads are monitored with hardware counters and subtracted from the memory latency measurements. The Mess benchmark release [32] includes the source code and its detailed description.

B. Validation

The Mess benchmark exceeds the coverage of all existing memory benchmarks and tools. Still, these tools can validate some of the Mess measurements.

The unloaded memory system latency can be measured with LMBench and Google multichase in CPU platforms, and P-chase [53] in GPUs. We used these benchmarks to validate the Mess unloaded latency measurements in all hardware platforms under study. In all experiments, Mess closely matches the LMBench, Google multichase and P-chase results.

In Intel systems, the maximum sustained memory bandwidth can be measured with the Intel Advisor [4]. In the Skylake, Cascade Lake and Sapphire Rapids servers under study, the Mess benchmark matches the Advisor measurements, with a difference below 1%.

The Intel Memory Latency Checker (MLC) [11], can measure the memory latency for a selected memory traffic intensity, i.e. memory bandwidth. The memory bandwidth can be fine-tuned, but the tool provides a sparse analysis of different traffic compositions, i.e. read and write memory operations. We compare the Intel MLC results and the

¹Memory traffic with more than 50% of writes can be generated with streaming stores that directly write data to the main memory. The public Mess repository already includes the benchmark with x86 streaming (non-temporal) stores. We are working on the equivalent benchmark version for ARM and Power CPUs (zero cache line fill instructions), and NVIDIA GPUs.

corresponding subset of the Mess measurements for all Intel platforms under study. The MLC and Mess results show the same trend, with slightly lower (<5%) latencies reported by the Mess benchmark. This is because the Mess is designed for close-to-the-hardware memory characterization, and, unlike the MLC, it excludes the latency penalties introduced by the OS overheads, the TLB misses and the page walk.

C. Performance analysis

Figure 2 illustrates the Mess benchmark performance analysis with an example of the 24-core Intel Skylake server with six DDR4-2666 memory channels [54]. The figure confirms a general trend of the memory access latency, which is initially roughly constant and then increases with higher memory pressure (i.e. bandwidth) due to resource contention among parallel accesses [37], [55], [56].

Detailed and close-to-the-hardware Mess characterization reveals some memory system aspects not discussed by previous studies. The most important one is the impact of the read and write memory traffic. The best performance, the lowest latency and the highest achieved bandwidth, are obtained for 100%-read traffic. Memory writes reduce the memory performance and reach the saturation point sooner. This is due to the extra timing constraints such as t_{WR} and t_{WTR} , which come with memory write operations [57]. We detect this behavior for all the Intel, AMD, IBM, Fujitsu, and Amazon servers as well as NVIDIA GPUs used in the study with DDR4, DDR5, HBM2 and HBM2E memory (Section III). However, we see a very different write traffic impact on CXL memory expanders [34]. This behavior of CXL memory expanders is analyzed in Section V-C.

Apart from the memory bandwidth–latency curves, we use the Mess benchmark to derive memory system performance metrics, also depicted in Figure 2, for quantitative comparison of different memory systems. In addition to the commonly-used unloaded memory latency, the detailed Mess characterization quantifies the **maximum latency range** of memory access latencies for all read/write ratios and the **saturated bandwidth range**. The memory system is saturated when any further increase in the memory system pressure, i.e. bandwidth, leads to a high increase in the memory access latency. We consider that the saturated bandwidth area starts at the point in which the memory access latency doubles the unloaded latency. In some of the platforms under study, we also detect that the increase in the Mess memory access rate causes a memory bandwidth decline, while the access latency continues to increase. This behavior is observed in the bandwidth–latency curves as a “wave form” seen in Figure 2. The causes for this memory behavior are analyzed in Section III. To the best of our knowledge, our study is the first one to detect and analyze this suboptimal memory system behavior.

Figure 2 also depicts the bandwidth reported by the **STREAM benchmark** (vertical dashed lines). STREAM and Mess provide complementary memory bandwidth analysis. While STREAM is the *de facto* standard for measuring application-level sustained memory bandwidth [5], [58], the

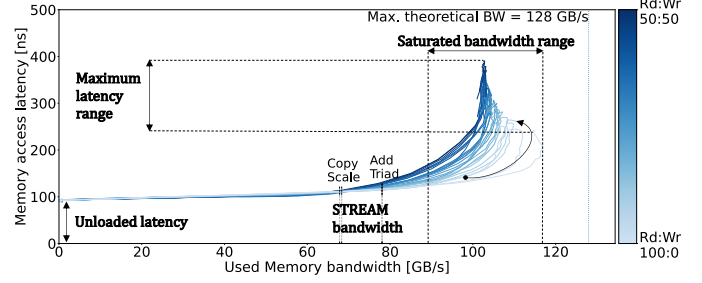


Fig. 2: The Mess benchmark models the memory system performance with a family of bandwidth–latency curves.

Mess benchmark provides the microarchitecture view and considers all memory traffic visible by hardware counters. Differences between these two approaches are analyzed in the next section.

III. PERFORMANCE CHARACTERIZATION: ACTUAL SYSTEMS

We use the Mess measurements to compare the memory system performance of Intel, AMD, IBM, Fujitsu and Amazon servers as well as NVIDIA GPUs with DDR4, DDR5, HBM2 and HBM2E. The platform and memory system characteristics are listed in Table I while Figure 3 shows their bandwidth–latency curves. The **unloaded memory latency** varies significantly between different platforms. It ranges from 85 ns in the Cascade Lake server with DDR4, to 122 ns in the A64FX servers with HBM2 and 129 ns in the Graviton 3 with DDR5 main memory. This difference should not be directly associated to the main memory. For example, the AMD Zen2 comprises DDR4 technology with practically the same command latencies (in nanoseconds) as the Intel Cascade Lake server, and still it shows almost 30 ns higher unloaded memory latency. This is because the load-to-use latency considers the time memory requests spend within the CPU chip, including the cache hierarchy and network on chip. These timings can differ significantly between different CPU architectures. We detect the highest unloaded memory latencies in the chips with the largest number of cores (Zen2, A64FX, Graviton 3) indicating that at least a portion of this latency is likely to be attributed to the network on chip which is larger and more complex in these architectures. NVIDIA H100 GPU shows higher unloaded latency due to its massive number of arithmetic processing units, lower on-chip frequency, and complex memory hierarchy [53], [65].

We also detect a wide **maximum latency range** between different platforms, and even within a single platform for different read and write memory traffic. Maximum memory latency is a primary concern in real-time systems, and it is less critical in high-performance computing (HPC). Still it has to be bound to guarantee quality of service of some HPC applications. We believe that our study may open a discussion about the sources of different maximum latencies in different systems, e.g. due to the different lengths of the memory queues, and about its desirable limits.

TABLE I: CPU and GPU platforms under study: Quantitative memory performance comparison.

Platform	Intel Skylake Xeon Platinum [54]	Intel Cascade Lake Xeon Gold [54]	AMD Zen2 EPYC 7742 [59]	IBM Power9 02CY415 [60]	Amazon Graviton 3 [61]	Intel Sapphire Rapids Xeon Platinum [62]	Fujitsu A64FX [63]	NVIDIA Hopper H100 [64]
Released	2015	2019	2019	2017	2022	2023	2019	2023
Cores @ frequency	24 @ 2.1 GHz	16 @ 2.3 GHz	64 @ 2.25 GHz	20 @ 2.4 GHz	64 @ 2.6 GHz	56 @ 2GHz	48 @ 2.2 GHz	132 SMs @ 1.1GHz
Main memory	6×DDR4-2666	6×DDR4-2666	8×DDR4-3200	8×DDR4-2666	8×DDR5-4800	8×DDR5-4800	4×HBM2	4×HBM2E
Theoretical bandwidth	128 GB/s	128 GB/s	204 GB/s	170 GB/s	307 GB/s	307 GB/s	1024 GB/s	1631 GB/s
% of the max theoretical bandwidth	72–91%	68–87%	57–71%	67–91%	63–95%	60–86%	72–92%	51–95%
Saturated bandwidth range	53–61%	51–57%	46–51%	32–36%	78–82%	63–66%	49–55%	64–69%
STREAM kernels: Application view								
Unloaded latency	89 ns	85 ns	113 ns	96 ns	122 ns	109 ns	129 ns	363 ns
Maximum latency range	242–391 ns	182–303 ns	257–657 ns	238–546 ns	332–527 ns	238–406 ns	338–428 ns	699–1433 ns

All platforms except AMD Zen2 CPU and NVIDIA H100 show a similar **saturated bandwidth range**, between approximately 70% and 90% of the maximum theoretical bandwidth. The maximum achieved bandwidth cannot reach the theoretical one because part of it is “lost” due to factors such as DRAM refresh cycles blocking the entire chip, page misses causing precharge and activate cycles, and timing restrictions at the bank, rank, and channel levels [66]. The efficiency of the memory bandwidth utilization also depends on the memory controller design. As expected (see Section II-C), the best utilization is achieved for 100%-read memory traffic, and it reduces with the increment of the memory writes. AMD Zen2 is an exception in two ways. First, its saturated bandwidth range is significantly lower, 57–71% of the maximum theoretical one. Second, it does not follow the expected impact of the write traffic on the bandwidth utilization. The traffic with the maximum rate of memory writes shows a very good performance, very close to the 100%-read traffic, while the main drop is detected for a mixed, e.g. 60%-read/40%-write, traffic.

In some AMD Zen2, Intel Skylake and Intel Cascade Lake bandwidth–latency curves, increase in the Mess memory access rate, after some point, leads to decline in the measured bandwidth.² In Amazon Graviton 3, Intel Sapphire Rapids and NVIDIA H100 this behavior is frequent for the memory traffic with high percent of writes. We explored these findings in the Cascade Lake servers in which we had access to the row-buffer hardware counters. In the experiments that experienced the bandwidth decline, we detect a significant increase in row-buffer misses. In case of a row-buffer miss, the current content of the row is stored in the memory array and the correct row is loaded into the row-buffer. These additional operations increase the memory access time and reduce the effective device bandwidth. To confirm these findings, we increased the memory system pressure by removing four out of six DIMMs in each socket. In these experiments, we detected a large “wave-form” segments in all bandwidth–latency curves. And indeed, the measurement confirmed that the measured bandwidth drop is highly correlated with the row-buffer miss rate.

The results also show the **difference between the STREAM and Mess measurements**. STREAM benchmark reports four data-points: the maximum sustained bandwidth of Copy, Scale, Add and Triad kernels. Mess provides a wide range of measurements for different memory traffic intensity and

²These findings are consistent with two recent studies which report that running high-bandwidth benchmarks on all CPU cores may lead to lower memory bandwidths w.r.t. the experiment in which some cores are not used [6], [67].

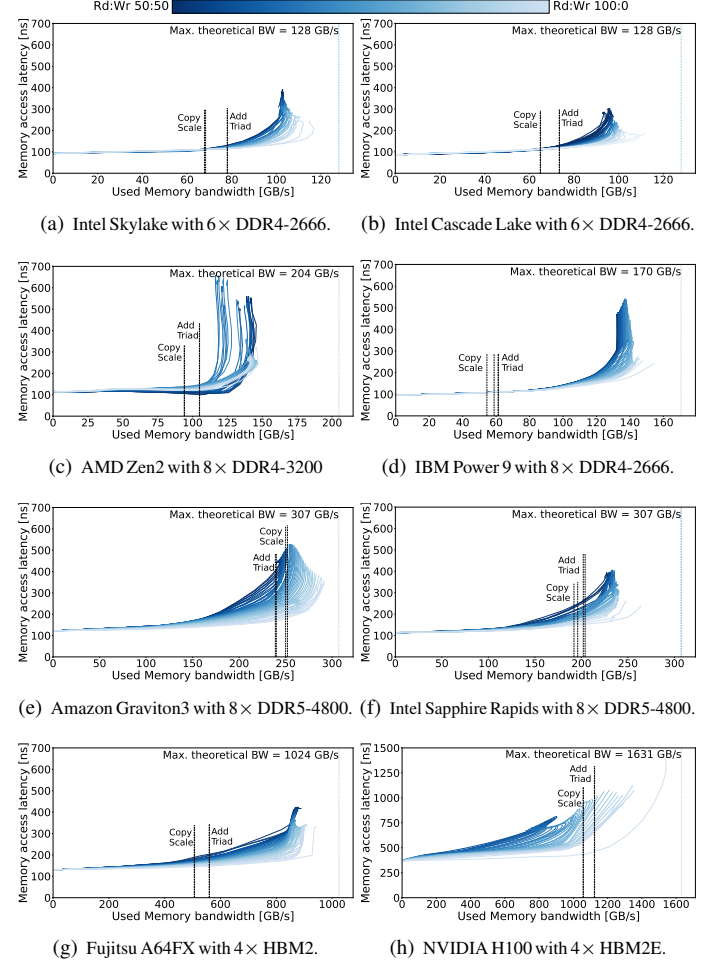


Fig. 3: We detect a wide range of memory system behavior, even for the hardware platforms with the same memory standard.

read/write ratios. STREAM measures the application-level data traffic estimated based on the application execution time, size of the data structures, and number of load and store instruction in each STREAM kernel. Mess measures the architecture-level memory bandwidth measured with hardware counters. This includes all the application data traffic, but also all the microarchitecture data transfers. For example, STREAM bandwidth calculation assumes one memory read for each load instruction and one memory write for each store. In state-of-the-art HPC servers with write-allocate cache policy, each store instruction does not correspond to a single memory

write, but to one read and one write (see Section II-A). For this reason, Mess benchmark reports higher maximum measured bandwidths. This difference is clearly visible in IBM Power9, A64FX and all Intel platforms. In Graviton3 and NVIDIA H100, the results reported by STREAM are very close to the maximum Mess measurements for the corresponding read/write ratio. This would correspond to a architecture with a write-through cache policy. The second cause of the difference between the maximum Mess and STREAM bandwidths is the memory traffic composition. The Mess benchmark achieves the maximum bandwidth for a 100%-read traffic. The write memory traffic, present in all STREAM kernels, adds timing constraints and reaches sooner the bandwidth saturation point. Our results show that different platforms show distinct relation between the application-level STREAM and architecture-level Mess measurements. Therefore, we believe that the memory bandwidth analysis should include both approaches. As a part of future work, we plan to compare the STREAM and Mess measurements in high-end architectures that dynamically adjust the level of write-allocate traffic [68], [69].

IV. PERFORMANCE CHARACTERIZATION: MEMORY SIMULATORS

Mess benchmark can also be used to characterize memory simulators and compare them with the actual systems they intent to model. We illustrate this Mess capability with the gem5, ZSim, and OpenPiton Metro-MPI simulators with different internal memory models and widely-used external memory simulators, DRAMsim3, Ramulator and Ramulator 2.

A. gem5

The gem5 [16] is a cycle-accurate full-system simulator. In our experiments, the simulator is configured to model the Graviton 3 server with 64 Neoverse N1 cores [61]. The cache hierarchy includes 64KB of 4-way L1 instruction and data cache, 1 MB of 8-way private L2 cache and 64 MB of 16-way shared L3. The main memory system has eight DDR5-4800 memory channels. Figure 4 compares Mess bandwidth–latency curves of the actual server with **gem5 simple memory model**, more complex **gem5 internal DDR model** and gem5 connected to **Ramulator 2**. To maintain a reasonable simulation time, we model each system with a family of six curves, from 50% to 100% read memory traffic with a 10% step.

Practically in the whole bandwidth range, the gem5 **simple memory model** delivers a fixed latency of 4–49 ns. The latency increases only when the bandwidth asymptotically approaches its theoretical maximum. Contrary to the Graviton 3 server, the highest latencies are measured for a 100%-read traffic, and the latency drops with the percent of memory writes. Also, unlike in the actual platform, for some memory traffic, increasing the bandwidth reduces the memory access latency. For example, the 50%-read/50%-write traffic reaches the lowest simulated latency of only 4 ns at the 200 GB/s bandwidth. The same traffic in the actual system has the memory access latency of 261 ns.

The more detailed **internal DDR model** shows small improvements over the simple memory model, but still poorly

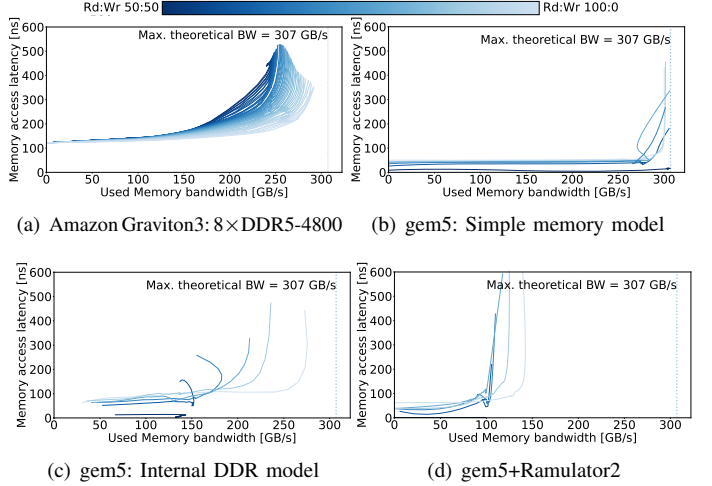


Fig. 4: Memory performance: Amazon Graviton3 server vs. gem5 memory models.

resembles the actual system performance. The simulated latencies are unrealistically low, most of them in the range of 14–100 ns. Similarly to the gem5 simple memory model, the latencies drops with the percent of memory writes. For all the curves except 100%-read, the saturated bandwidth is significantly lower from the one measured on the actual system. Again, the error increases with the percent of memory writes.

As the internal gem5 memory models, **gem5+Ramulator 2** simulates unrealistically low memory latencies and the error increases with the ratio of memory writes. In addition to this, the curves experience a sharp, nearly vertical rise between 100 GB/s and 130 GB/s, which is less than a half of the actual measured bandwidth. Surprisingly, the most complex and trusted memory model shows the highest simulation error. Some sources of this error will be analyzed in Section IV-D.

B. ZSim

We select ZSim [15] as a representative of event-based hardware simulators. We use publicly-available ZSim modeling 24-core Intel Skylake processor connected to six DDR4-2666 channels [70]. The cache hierarchy of the modeled CPU includes 64KB of 8-way L1 instruction and data cache, 1 MB of 16-way private L2 cache and 33 MB of 11-way shared L3. The simulator is extensively evaluated against the actual hardware platform [71]. The ZSim comprises three internal memory models: fixed-latency, M/D/1 queue model and the internal DDR model. Also it is already connected to Ramulator [23] and DRAMsim3 [22]. To avoid any simulator integration error, we use the ZSim+DRAMsim3 released by the University of Maryland (DRAMsim3 developers) [72], and ZSim+Ramulator from The SAFARI Research Group at the ETH University (Ramulator developers) [73].

Figure 5 compares the Mess bandwidth–latency curves of the actual server with **all five ZSim memory simulation approaches**. As expected, the **fixed-latency memory model** provides a constant latency in the whole bandwidth domain.

Given that this latency is configured by a user, it can be set to match the unloaded memory latency in the actual system. On the down side, the memory bandwidth provided by this model is unrealistic: the maximum simulated bandwidth is 342 GB/s, which exceeds the maximum theoretical one by $2.7\times$. The **M/D/1 queues** correctly model the memory system behavior in the linear part of the curves. The modeling of the system saturation is less accurate. The queue model does show some difference between read and write memory traffic, but the reported performance does not correspond to the actual system trend in which increasing the write traffic lowers the performance. The **internal DDR model** correctly emulates the linear and saturated segments of the curves and the impact of the memory writes. However, the simulator underestimates the saturated bandwidth area to 69–93 GB/s, significantly below the 92–116 GB/s measured in the actual system. Also, the simulator excessively penalizes the memory writes which is seen as a wider spread of the curves with a higher write memory traffic. Finally, we detect some unrealistic memory-latency peaks in the low-bandwidth 1–4 GB/s curve segments. The **DRAMsim3** shows a similar trend as the M/D/1 queue model in the linear segments of the memory curves, with some latency error, 52–63 ns in the DRAMsim3 versus 89–109 ns in the actual system. The simulator does not model the saturated bandwidth area. Finally, the **Ramulator** provides a fixed 25 ns latency in the whole bandwidth area and for all memory traffic configurations. Also, similar to the fixed-latency model, the simulated bandwidth is unrealistic, exceeding by $1.8\times$ the maximum theoretical one.

Our evaluation of the memory models and detailed hardware simulators detected major discrepancies w.r.t. the actual memory systems performance. DRAMsim3, Ramulator and Ramulator 2 are considered *de facto* standard for the memory system simulation. All of them are evaluated against the manufacturer’s Verilog model and they show no violation of the JEDEC timings [57], [74]. DRAMsim3 is evaluated for DDR3 and DDR4 [22], Ramulator for DDR3 [23], and Ramulator 2 for DDR4 [24]. However, as our results demonstrate, this does not guarantee that the simulators properly model the memory system performance. In Section IV-D, we will use Mess benchmark to analyze some causes of these discrepancies.

C. RTL simulators: OpenPiton and Metro-MPI

OpenPiton framework [75] provides an open-source RTL implementation of a tiled architecture based on Ariane RISC-V cores [76]–[78]. Developed in the Verilog RTL, the OpenPiton simulation is slow, especially for large number of cores. We use the OpenPiton simulation accelerated by Metro-MPI [33]. This approach uses Verilator [79] to convert the RTL code of each tile into a cycle-accurate C++ simulation model. Then, all the tiles are simulated in parallel and their interconnect communication is done with the MPI programming interface.

In our experiments, the OpenPiton framework is configured to generate 64-core Ariane architecture which includes 16 KB of 4-way L1 instruction and data cache, and 4 MB of 4-way shared L2 cache. The main memory is originally modeled

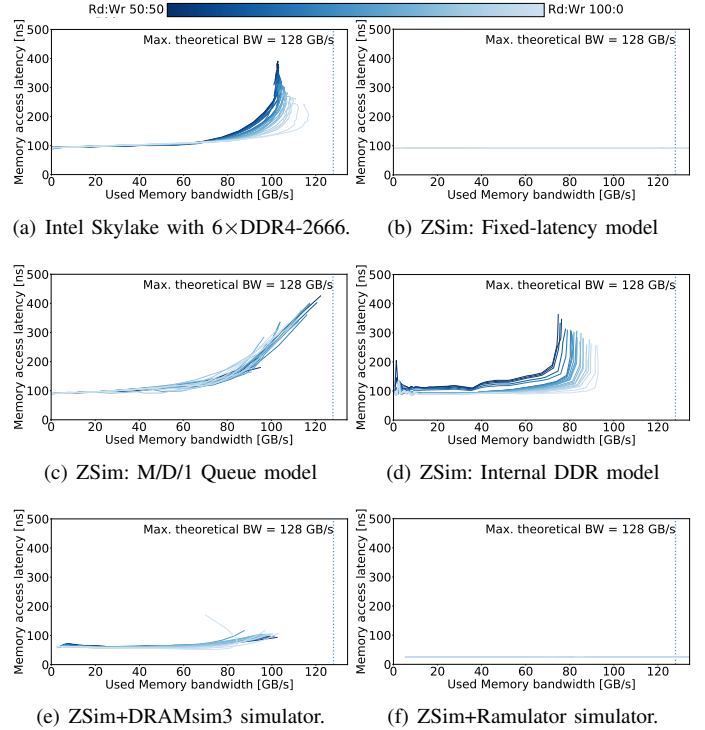


Fig. 5: Memory performance: Intel Skylake server vs. ZSim memory models.

with a single-cycle latency, and it is recently extended with a fixed-latency model [77]. Our Mess measurements confirm that both models deliver the expected load-to-use latency. Also, as expected, we see no difference between read and write memory traffic, leading to a perfect overlap of the curves. The only difference is in the maximum observed memory bandwidth. For a single-cycle memory latency, 100%-read memory traffic achieves 32 GB/s, limited by the memory concurrency of the 64 in-order Ariane cores. Memory writes do not stall the cores, so the achieved memory bandwidth increases with the write memory traffic ratio. Still, a small 2-entry miss status holding registers (MSHR) limits the memory bandwidth to 47 GB/s for 50%-read/50%-write traffic. We detect the same trend for the fixed memory model.

The Mess evaluation of the OpenPiton Metro-MPI resulted in an unexpected discovery: in some experiments we detected significantly higher memory write traffic than anticipated. By analyzing the system behavior for various Mess configurations, we connected the extra memory traffic to the unnecessary eviction of the data from the last-level cache. Instead of evicting only the dirty cache lines, the system was evicting all of them. The source of the error is the coherency protocol generated by the OpenPiton framework. The error was reported to the OpenPiton developers and they confirmed its existence.

D. Sources of memory simulation errors

To exclude any simulation error caused by the CPU simulators or their memory interfaces, we perform a **trace-driven**

DRAMsim3, Ramulator and Ramulator 2 simulation. The detailed Mess memory traces are collected from its ZSim simulation, and they include the addresses of all memory read and write operations. To account for the timings of non-memory operations, DRAMsim3 traces contain simulation cycles in which the memory requests reach the memory controller. The Ramulator and Ramulator 2 traces include the number of non-memory instructions between the consecutive memory operations. The Mess traces may contain some timings errors w.r.t. actual execution, so the DRAMsim3, Ramulator, and Ramulator 2 simulations may not match the exact bandwidth–latency point. Still, the correct memory simulation should provide data-points that are located on the actual bandwidth–latency curves. Figure 6 shows the trace-driven Mess evaluation of DRAMsim3, Ramulator and Ramulator 2. The charts report the round-trip memory access latency from the memory controller, so it is expected that the simulated curves are somewhat below the actual load-to-use measurements.

The trace-driven **Ramulator 2** show the same bandwidth–latency trends as the gem5+Ramulator 2 simulations (see Figure 4(d)). The simulated memory latency is unrealistically low and the maximum simulated memory bandwidth is only 126 GB/s which is less than a half of the 292 GB/s measured in the actual system. This indicates that the main source of the large simulation error is indeed Ramulator 2.

The conclusions are somewhat different for **DRAMsim3** and **Ramulator** as their trace-driven memory bandwidth–latency curves show better general trends than the corresponding Zsim-driven simulations. This indicates that a part of the ZSim+DRAMsim3 and ZSim+Ramulator simulation errors reported in Figure 5 is caused by the simulators’ interfaces. Our finding is aligned with the previous studies that report issues in the integration of the event-based CPU simulators with cycle-accurate memory models [49], [80], [81].

However, trace-driven DRAMsim3 and Ramulator also show important discrepancies w.r.t. actual bandwidth–latency curves. **DRAMsim3** simulated latency starts at 68 ns. Apart from the peak at 5 GB/s, the latency increases linearly with the bandwidth. The curves for different read/write ratios are spread and intertwined in whole bandwidth range. Below 70 GB/s the curves with the highest write traffic ratio have the lowest latency. We detect no bandwidth saturation, and all the curves linearly reach the maximum bandwidth of 113 GB/s. **Ramulator** shows a better general trend: roughly-constant latencies below 40 GB/s, a light latency increase until approximately 85 GB/s, and a higher inclination in the final segments of the curves. Still, the latency is unrealistically low, starting at only 25 ns (100%-read traffic, 20–40 GB/s), different read/write curves are spread in all bandwidths, and the saturated behavior differs significantly from the actual one.

To understand better the underlying causes of the trace-driven DRAMsim3 and Ramulator simulation errors, we compare their **row-buffer hit, empty and miss statistics** with the measurements from the actual Intel platform. We could not perform the same analysis for the Ramulator 2 because its baseline architecture, Amazon Graviton3 with 8×DDR5-4800,

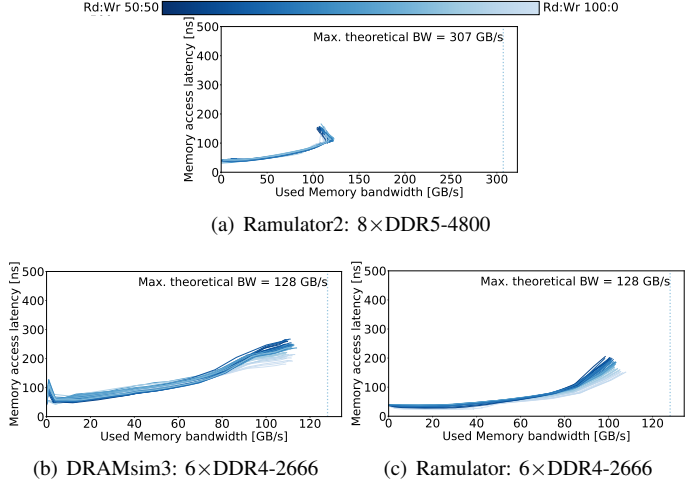


Fig. 6: Memory performance: Trace-driven cycle-accurate simulators.

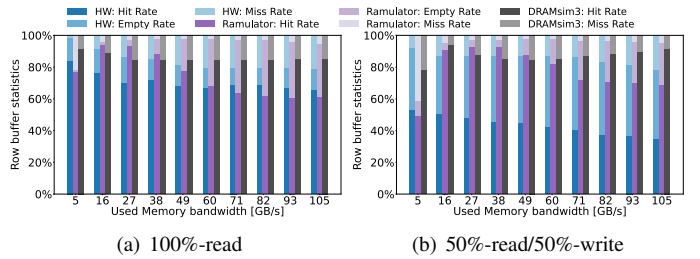


Fig. 7: Row buffer statistics: Actual hardware vs. DRAMsim3 vs. Ramulator.

does not support the row-buffer measurements. Figure 7 shows a subset of the results for the 100%-read and 50%-read/50%-write memory traffic, which is sufficient to show a general trend. For a 100%-read traffic and low memory bandwidth utilization, the actual system has 84% row-buffer hits, 13% empty buffers and 3% misses. As expected, higher memory bandwidth utilization decreases the hit ratio, and increases the empty pages and misses. Also, as we increase the write traffic, the row-buffer utilization degrades, compare Figure 7(a) and 7(b).

DRAMsim3 shows a very different behavior. In most of the experiments, we measure 84–93% row-buffer hit rate with 7–16% of the misses. We detect the highest hit-rates for the dominantly-read and dominantly-write traffic, while intermediate read/write ratios have lower values. These hit-rates match the vertical spread we see in the DRAMsim3 bandwidth–latency curves for low and medium bandwidths (80 GB/s). The curves with high hit-rates have the lowest latencies. For the curves with lower hit-rates, the latency increase by up to 20 ns. In 2 GB/s DRAMsim3 experiments some of the read/write ratios have a surprisingly low row-buffer hit-rates (< 35%), perfectly matching the Figure 6(b) memory latency peak at this bandwidth data-point. The **Ramulator** row-buffer statistics resemble better the actual measurements, see Figure 7. Still we detect some discrepancies that, interestingly, have similar

trends as the DRAMsim3 results. Again, we detect the highest hit-rates for the dominantly-read and dominantly-write traffic, while intermediate read/write ratios have lower values. For $> 40\%$ write traffic, Ramulator hit-rates greatly exceed the actual ones in the whole bandwidth range (Figure 7(b)). As in DRAMsim3, these hit-rates closely match the vertical spread of the Ramulator latency simulations.

Comparison of the **actual platform** row-buffer statistics and the latency measurements resulted in an interesting finding. For low and moderate bandwidth utilization (< 70 GB/s), the write traffic increase leads to a notably worse row-buffer utilization, but this does not translate into higher memory access latencies. It seems that the actual system is capable to mask the row-buffer contentions and delays. We do not see this behavior in the DRAMsim3 and Ramulator simulations. Overall, our analysis of the row-buffer statistics and its correlation with the bandwidth–latency curves provides some first steps in the analysis of the memory simulation errors. We believe that our work and publicly-released Mess benchmark will motivate the community to continue this exploration.

The memory access pattern has a significant impact on the row-buffer utilization and the overall memory system performance. The Mess benchmark concurrently executes random-access pointer-chase and the multi-process memory traffic generator. Each process traverses two arrays, one with load and one with store operations. Each array is accessed sequentially, but, the overall memory access pattern is complex due to the concurrent traversal of tens of distinct arrays located in main memory. Mess traffic generator can be easily extended to cover different array access patterns. Some of these patterns are strided access, e.g. targeting a new row-buffer in each operation, or the random access, e.g. the RandomAccess test in HPC Challenge benchmark developed to measure Giga Updates Per Second (GUPS) in a system [82].

V. MESS SIMULATOR

In this section we will present the Mess analytical memory system simulator and show how it significantly improves the memory simulation accuracy and enables a quick adoption of new memory technologies in hardware simulators.

A. Design

The CPU and memory simulators are typically connected in the following way: the CPU simulator issues the memory operations and the memory simulator determines their latencies. The Mess simulator does this analytically based on the application’s position in the memory bandwidth–latency curves. This process is complex due to the inherent dependency between the memory system latency, timings of the memory operations and all dependent instructions, and the generated memory bandwidth. We simplify the problem by designing the Mess simulator not to compute the exact memory latency for a given memory traffic, but to detect and correct discrepancies between the memory access latency and the simulated bandwidth. This approach, together with the fundamental

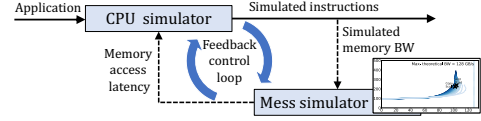


Fig. 8: Mess feedback control loop adjusts the simulated memory access latency.

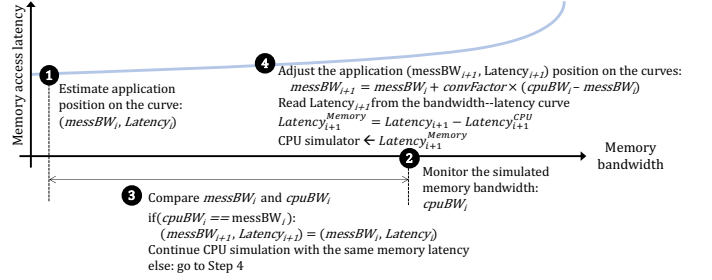


Fig. 9: One control loop iteration: Mess simulator monitors the simulated memory bandwidth, $cpuBW_i$, and compares it with the $messBW_i$ estimated at the beginning of the simulation window. If a major difference is detected, the Mess simulator adjusts the application position in the bandwidth–latency curves.

principle of application’s position in the memory bandwidth–latency curves, enables the Mess simulator to surpass the accuracy of all other memory simulators, while remaining simple and fast. The only memory system parameter required by the Mess model is the family of the bandwidth–latency curves. The curves can be measured on the actual hardware (Section III) or can be provided by the manufacturers, e.g. based on their detailed hardware model, as we will discuss in Section V-C.

The Mess simulator acts as a feedback controller [83] from the classical control theory [84], illustrated in Figure 8. The simulation can start from any memory access latency, e.g. the unloaded one. This latency is used by the CPU simulator which generates memory memory reads and writes. The Mess simulator observes this simulated memory bandwidth, positions it at the corresponding memory bandwidth–latency curve, and controls whether it coincides with the memory latency used in the CPU simulation. If this is not the case, the memory latency is being adjusted with an iterative process we describe later.

The control process is performed at the end of each simulation window, which, in our experiments, comprises 1000 memory operations. This is much smaller than the length of the application phases [66], so the transition error between different application phases has a negligible impact on the simulator’s accuracy.

Figure 9 describes one iteration of the Mess simulator control loop. For simplicity, the figure shows a single bandwidth–latency curve. We start with the Mess estimate of the application’s bandwidth–latency position in the i^{th} simulation window, $(messBW_i, Latency_i)$ ①. The curve is selected based on the read/write ratio of the simulated memory traffic,

and the x -axis position is determined by $messBW_i$. From that point on, all the issued memory requests are simulated with $Latency_i$. At the end of the simulation window, the Mess simulator monitors the simulated memory bandwidth, $cpuBW_i$ ②, and compares it with the $messBW_i$ estimated at the beginning of the window ③.

If the simulation is in a steady-state and the application did not change its behavior, there will be no major difference between $cpuBW_i$ and $messBW_i$. This confirms the consistency in the simulated memory access latency, the CPU timings and the achieved memory bandwidth. Therefore, the CPU simulation in the next window will continue with the same memory latency.

Otherwise, a difference between $cpuBW_i$ and $messBW_i$ suggests inconsistent simulated memory latency and bandwidth. This can happen, for example, if the application changes its behavior. Figure 9 illustrates the case in which the application increases the frequency of memory request leading to the higher bandwidth: $cpuBW_i > messBW_i$ ③. In this case, the simulated memory bandwidth $cpuBW_i$ does not correspond to the memory $Latency_i$ used in the CPU simulation. To address this inconsistency, the Mess simulator adjusts the predicted application position in the bandwidth–latency curves. The objective of this adjustment is not to reach the correct $(BW, Latency)$ position in a single iteration. The next Mess estimate, $(messBW_{i+1}, Latency_{i+1})$, will be positioned in-between $messBW_i$ and $cpuBW_i$ ④. The exact position is determined based on the user-defined convergence factor: $messBW_{i+1} = messBW_i + convFactor \times (cpuBW_i - messBW_i)$. The approach is based on the proportional–integral controller mechanism from the control theory [83], [84]. Finally, the Mess uses $messBW_{i+1}$ to read the $Latency_{i+1}$ from the corresponding bandwidth–latency curve. The $Latency_{i+1}$ is a load-to-use latency. This includes the time spent in the CPU cores, cache hierarchy and network on chip, already considered in the CPU simulation. In the final step, the $Latency_{i+1}$ is adjusted by these CPU timings $Latency_{i+1}^{Memory} = Latency_{i+1} - Latency_{i+1}^{CPU}$. The next simulation window starts with the Mess providing the updated $Latency_{i+1}^{Memory}$ to the CPU simulator.

B. Evaluation

The Mess simulator is integrated with ZSim and gem5, and evaluated against the actual hardware. We compare the simulated and actual bandwidth–latency curves as well as the performance of memory-bound benchmarks: STREAM [5], LMBench [8], and Google multichase [9].

1) *ZSim*: Figure 10 shows the DDR4, DDR5 and HBM2 bandwidth–latency curves measured with the ZSim connected to the Mess simulator.³ The configurations of the simulators match the actual Intel Skylake with 24-core and six DDR4-2666 memory channels. The simulated Mess curves, depicted

³The Mess simulator also supports the Intel Optane technology. Optane’s bandwidth–latency curves are measured on a 16-core Cascade Lake server with 6×DDR4-2666 16GB and 2×Intel Optane 128GB memory in App Direct mode. Intel Optane technology is discontinued since 2023, so we do not analyze its performance characteristics and simulation.

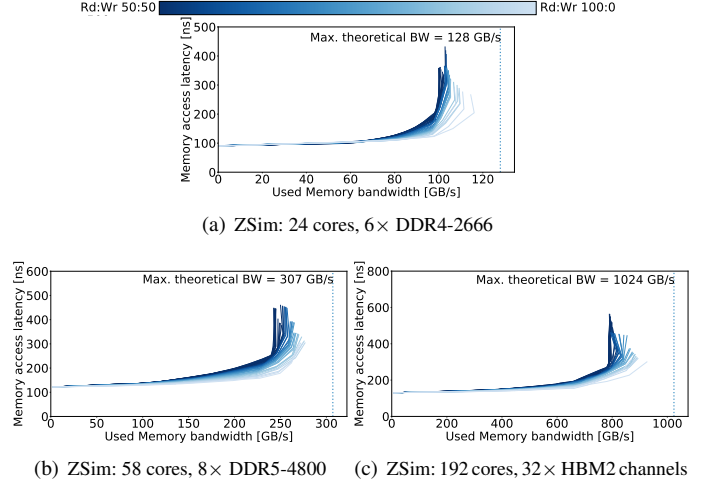


Fig. 10: ZSim with the Mess simulator closely matches the actual memory systems.

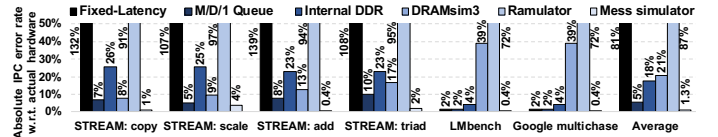


Fig. 11: The ZSim+Mess simulation error for STREAM, LMBench and Google multichase is only 1.3%.

in Figure 10(a), closely resemble the actual memory systems performance (Figure 3(a)). The simulation error of the unloaded memory latency is below 1%, and it is around 3% for the maximum latencies. The difference between the simulated and the actual saturated bandwidth range is only 2%. Figures 10(b) and 10(c) show the ZSim+Mess simulation results for the high-end DDR5 and HBM memories. To saturate the 8-channel DDR5-4800 and 32-channel HBM2, we increase the number of simulated cores to 58 and 192, respectively. Again, the simulated bandwidth–latency curves closely resemble the performance of the corresponding actual memory systems (Figures 3(g) and 3(e)).

Figure 11 shows the evaluation results, w.r.t. to the actual Intel Skylake server, of all six ZSim memory models when running memory intensive STREAM [5], LMBench [8] and Google multichase [9]. The simulation errors are closely correlated with the similarity between the simulated and actual bandwidth–latency curves. The Mess shows the best accuracy with only 1.3% average error, followed by the M/D/1 and internal DDR model. The fixed-latency simulation and Ramulator show the highest errors of more than 80%. The Mess simulator is also fast. It increases the simulation time by only 26% higher w.r.t. a simple fixed-latency memory, and it is 2% and 15% faster than the M/D/1 and internal DDR model. The ZSim+Mess simulation speed-up over the ZSim+Ramulator and ZSim+DRAMsim3 is 13× and 15×.

2) *gem5*: Figure 12 shows the DDR5 and HBM2 bandwidth–latency curves simulated with the gem5 connected

to the Mess memory simulator. In all experiments, the gem5 is configured to model Graviton3 cores [61] described in Section IV-A. To reduce simulation time, we simulate 16 CPU cores connected to a single memory channel.⁴ The simulated bandwidth–latency curves, when scaled to eight DDR5 channels or 32 HBM2 channels, closely resemble actual system behavior (Figures 3(e) and 3(g)).

We also evaluate the Mess memory simulation against the gem5’s built-in simple memory model and internal DDR5 model as well as cycle-accurate Ramulator2 when running STREAM, Lmbench, and Google Multichase benchmarks. In these experiments we simulate a whole server comprising 64 Graviton3 cores and 8×DDR5-4800, and compare the results against the benchmark executions on the actual server. The gem5+Mess simulation time is practically the same as the gem5 internal DDR model, while it provides much better accuracy. The Mess memory simulator decreases the average error from 30% (gem5 simple memory model), 15% (internal DDR5 model), and 52% (Ramulator 2) to only 3%. Such a low error is unprecedented in any prior validation attempts [85]–[88].

C. Simulation of novel memory systems: CXL memory expanders

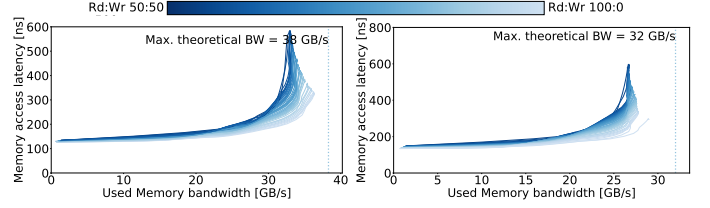
The memory system complexity and the scarcity of publicly-available information often result in a considerable gap between a technology release and the support for its detailed hardware simulation. For example, public memory simulators started to support the DDR5 in 2023 [24], three years after production servers with DDR5 DIMMs hit the market.

The Mess simulator provides a fundamental solution for this gap because it can simulate emerging memory systems as soon as their bandwidth–latency curves are available. For memory technologies available on the market, the curves can be measured on a real platform. For emerging memory devices that are not yet available in off-the-shelf servers, the bandwidth–latency curve can be measured on a developer board with a prototype of the new device, or alternatively it can be provided by the manufacturers, e.g. based on their detailed proprietary RTL models.

We will demonstrate the Mess simulation of novel memory systems with an example of the Compute Express Link (CXL) memory expanders. CXL is an emerging interconnect standard for processors, accelerators and memory devices. The CXL memory expanders enable a straightforward enlargement of the memory system capacity and bandwidth, as well as the exploration of unconventional disaggregated memory systems [34]. One of the main limitations for an academic research in this field, however, is the lack of reliable performance models for these devices.

The Mess simulation is performed with the CXL memory expanders bandwidth–latency curves provided by the memory manufacturer based on their detailed hardware model. In particular, we model a CXL memory expander connected to

⁴Simulation of a whole server comprising 64 Graviton3 cores and 8×DDR5-4800 requires more than five hours to obtain a single bandwidth–latency datapoint. The full simulation of all the curves would require more than a year.



(a) gem5: 16 cores & 1× DDR5-4800 (b) gem5: 16 cores & 1× HBM2 channel

Fig. 12: gem5 with the Mess simulator, when scaled, closely follows the actual memory system curves (Figures 3(e) and 3(g)).

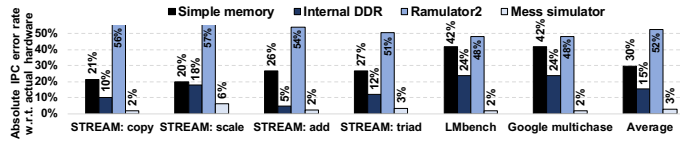
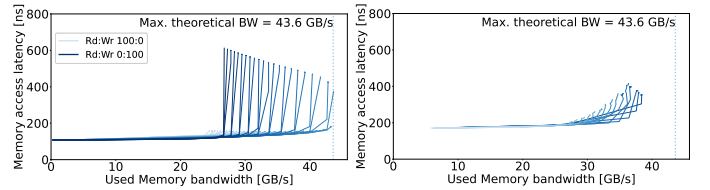
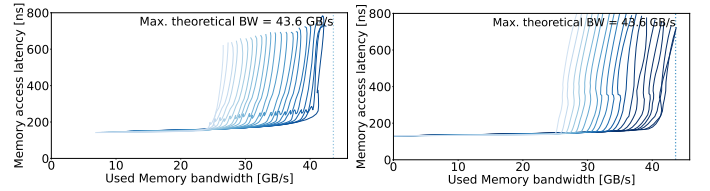


Fig. 13: The gem5+Mess simulation error for STREAM, Lmbench and Google multichase is only 3%.



(a) Manufacturer's SystemC model (b) OpenPiton: Ariane 64-core CPU



(c) gem5: Amazon Graviton3 16-core CPU (d) ZSim: Intel Skylake 24-core CPU

Fig. 14: Bandwidth–latency curves of the CXL memory expander. Mess simulator closely follows the Manufacturer's SystemC model.

the host via the CXL 2.0 PCIe 5.0 interface with 1×8 Lanes. The device comprises one memory controller connected to a DDR5-5600 DIMM with two ranks. All the CXL modules, Front end, Central controller and Memory controller, are implemented in SystemC. The modules communicate by using the manufacturer's proprietary SystemC Transaction Level Modeling (TLM [89]) framework,

The obtained bandwidth–latency curves are shown in Figure 14(a).⁵ The figure plots the round-trip latency from the CXL host input pins. To consider a full load-to-use latency, a user should add the round-trip time between the CPU core and

⁵The maximum theoretical bandwidth of the CXL.mem protocol is influenced by the read/write ratio of the workload [90]. In this figure, we present the highest value among all possible scenarios.

the CXL host. We measure this latency component with the Intel MLC [11]. The CXL memory expanders show a similar performance trends as the DDRx or HBM memory systems: latency that increases with the system load, significant non-linear increase after a saturation point, and the impact of the traffic read/write ratio. One major difference is that the CXL interface provides the best performance for a balanced reads and writes traffic, while its performance drops significantly for the 100%-read or 100%-write traffic. This is because, unlike the DDRx interfaces, CXL is a full-duplex interconnect with independent read and write links. Therefore, the CXL can transmit simultaneously in both directions, but in the case of the unbalanced traffic one CXL transmission direction could be saturated while other direction is negligibly used.

We use the obtained CXL memory bandwidth–latency curves in the Mess simulator integrated with ZSim, gem5 and OpenPiton Metro-MPI (see Figure 14). In all configurations, the Mess simulator closely follows the Manufacturer’s SystemC model. To reduce long OpenPiton Metro-MPI simulation time, we model only 25 curves with a small number of experimental points in each curve. For this reason some segments of the curves are discrete. Nevertheless, the OpenPiton Metro-MPI simulations match the general trend and the saturated bandwidth range of the manufacturer’s curves. The maximum latency range is below the manufacturers CXL curves because the simulated small in-order Ariane cores with only 2-entry MSHRs cannot saturate the target memory system. This behavior is already detected and discussed in Section IV-C. ZSim and gem5 results practically match the manufacturer’s CXL curves. In Appendix B, we compare our CXL simulation platform against prior approach to emulate memory-over-CXL.

VI. MESS APPLICATION PROFILING

The Mess framework also enhances the memory-related application profiling. We demonstrate this functionality with the Mess extension of Extrae and Paraver, production HPC performance tools for detailed application tracing and analysis [36]. The Mess application profiling adds a new layer of information related to the application’s memory performance metrics. This information can be correlated with other application runtime activities and the source code, leading to a better overall understanding of the application’s characteristics and behavior.

A. Background: Extrae and Paraver

Paraver is a flexible data browser for application performance analysis [91], [92]. It can display and analyze application MPI calls, duration of the computing phases, values of the hardware counters, etc. Paraver can also summarized application behavior in histograms and link it with the corresponding source code. The input data format for Paraver is a timestamped trace of events, states and communications [93]. For parallel applications, the traces are usually generated with the Extrae tool [94].

Extrae automatically collects entry and exit call points to the programming model runtime, source code references, hardware counters metrics, dynamic memory allocation, I/O system calls, and user functions. It is compatible with programs written

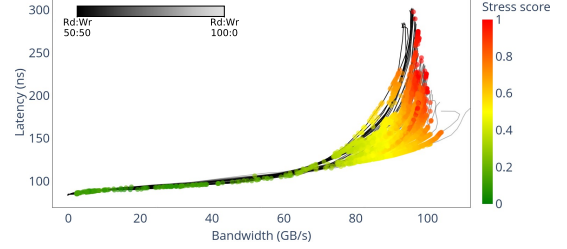


Fig. 15: Most of the HPCG execution is located in the saturated bandwidth area, above 75 GB/s. The Mess application-profiling extension of Paraver already includes the memory stress score visualization with a green–yellow–red gradient.

in C, Fortran, Java, Python, and combinations of different languages. It supports a wide range of parallel programming models. Extrae is available for most UNIX-based operating systems and it is deployed in all relevant HPC architectures, including CPU-based systems and accelerators.

B. Use cases

We illustrate the capabilities of the Mess application profiling with an example of the memory-intensive HPCG benchmark [7], [95] running in a dual-socket Cascade Lake server (Table I). We fully utilize the one CPU socket by executing 16 benchmark copies, one on each core. Extrae monitors the application memory behavior with a dedicated profiling process which traces the memory bandwidth counters. The sampling frequency is configurable, and it is 10 ms by default. Even with this fine-grain profiling, the introduced overhead is negligible, below 1%.

The extended Paraver tool correlates the application memory bandwidth measurements with Mess memory curves. The application measurements are plotted on the curves as a set of points, each of them corresponding to 10 ms of the application runtime. The application memory use can be also incorporated into the Paraver trace file, so a user can analyze its evolution over time, and correlate it with other application’s behavior and the source code.

1) *Bandwidth–latency curves*: Figure 15 depicts the Mess profile of the HPCG benchmark. Most of the HPCG execution is located in the saturated bandwidth area, above 75 GB/s. Sporadically, the benchmark even reaches the maximum sustained bandwidth with peak memory latencies in the range of 260–290 ns. Also, each HPCG point on the curves is associated with a memory stress score. The score value ranges from 0, for the unloaded memory system, to 1, corresponding to the right-most area of the bandwidth–latency curves. Memory stress score in a given point is calculated as a weighted sum of the memory latency and the curve inclination. The latency itself is a good proxy of the system stress, while the inclination shows the memory system sensitivity to a bandwidth change. Gentle inclination indicates that a memory bandwidth change would have a minor impact on the memory access latency and the overall performance. In the steep curve segments, e.g. 95–100 GB/s area in Figure 15, small bandwidth changes can rapidly saturate the memory system leading to a major latency

increase. The Mess extension of Paraver already includes the stress score visualization with a green–yellow–red gradient that can be easily interpreted by application developers.

2) *Timeline analysis*: Once the memory stress score is incorporated into the application’s Paraver trace, it can be combined with other aspects of the application analysis, as illustrated in Figure 16. The figure analyzes around two seconds of the HPCG runtime, from 241,748,818 μs to 243,728,242 μs (x -axis). Guided by the sequence of MPI calls illustrated in Figure 16 (top), we identify the application’s main iterative loop and, using MPI_Allreduce (pink) as delimiter, we select two iterations for our analysis. The middle Figure 16 analyzes compute applications phases. The color gradient corresponds to the compute phase length: green to blue gradient for short to long phases. Figure 16 (bottom) shows the memory stress score for this region. The longest compute phases (blue) exhibits two distinct memory behaviors: at the start of the phase, the memory stress score rises to 0.71, and then halfway through the phase it decreases to 0.64. The fine-grain application profiling can detect different memory stress score values even within a single compute phase.

3) *Links to the source code*: Extrae also collects callstack information of the MPI calls, referred to as the MPI call-points,⁶ which are used to link the application runtime behavior with the source code. With the Mess application-profiling extension of Paraver, the application source code can be linked to its memory-related behavior. This is fundamental for making data placement decisions in heterogeneous memory systems, e.g. comprising DDRx DIMMs and HBM devices [96], [97].

VII. RELATED WORK

Mess framework provides a unified view of the memory system performance that covers the memory benchmarking, simulation and application profiling. Although these three memory performance aspects are inherently interrelated, they are currently analyzed with distinct and decoupled tools.

A. Memory system benchmarks

Memory access latency and utilized bandwidth are commonly treated as independent concepts measured by separate memory benchmarks. LMBench [8] and Google Multichase [9] measure the load-to-use latency in an unloaded memory system, while STREAM [5], [98], STREAM2 [99], Hopscotch [100], CAMP [6], and HPCG [7], [95] measure the maximum obtainable memory bandwidth or performance of the application kernels that are proportional to it. Only recently the community started to make the first steps in measuring latencies in loaded memory systems. The Intel Memory Latency Checker (MLC) tool [11] is used to show how memory access latency increases for higher used memory bandwidths [27], [38] and to compare systems based on fundamentally different memory technologies, such as the DRAM and Optane [39]. X-Mem benchmark [12]

⁶A callpoint refers to the file and line number where the program initiates an MPI call at a given level of the callstack, typically the last one. This point serves as a boundary for a region that begins with the current MPI call and extends until the next one. In the tables, we indicate the starting callpoint.

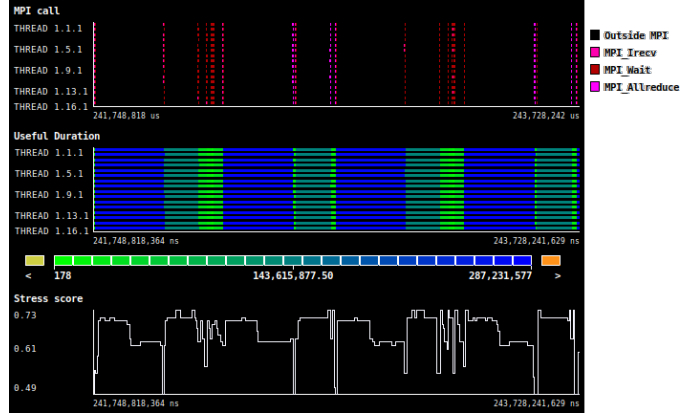


Fig. 16: Timeline showing two iterations of the HPCG benchmark with MPI calls (top), computations duration (middle) and memory stress score (bottom).

reports loaded access latencies for cache subsystem, main memory, and NUMA memory nodes. The impact of the read and write traffic to the memory system performance is not measured nor analyzed. Overall, current memory systems benchmarks provide a small number of data points in a very large and complex memory-performance space.

The Mess benchmark is designed for holistic close-to-the-hardware memory system performance characterization that is easily adaptive to different target platforms. It significantly increases the coverage and the level of detail of the previous tools, leading to new findings in memory behavior of the hardware platforms and simulators under study.

B. Memory system simulation

The Mess framework tightly integrates the memory performance characterization into the memory simulation. We compare the Mess analytical memory system simulator with the state-of-the-art memory models included in the CPU simulators [13], [14], [18], [49], [76]–[78], [86], [87] as well as the external cycle-accurate memory simulators [22]–[24]. The Mess memory simulator is fast, accurate and easily-integrated with the CPU simulators. It can support novel memory systems as soon as their detailed bandwidth–latency curves are measured on actual production systems or prototypes, or provided by the manufacturers. Therefore, Mess can simulate high-end and future memory systems much sooner than the standard memory simulators which consider detailed memory device sequences and timings [20]–[25]. Mess is the first memory simulator to support CXL memory expanders.

Apart from the hardware simulation, system performance can be analyzed with analytical models. The PROFET model [55] predicts how an application’s performance and energy consumption change when it is executed on different (future) memory systems. The model is based on the instrumentation of an application execution on actual hardware, so it already takes into account CPU microarchitectural details. PROFET is evaluated on Intel and Huawei servers with DDR3, DDR4, HBM and Optane memory [101]. The main objective is to

provide an alternative to complex and slow CPU simulations. This is orthogonal and complementary to the Mess simulator that targets the main memory simulation.

C. Application profiling

ProfileMe [26] and PerfMemPlus [27] determine whether the application is memory bound based on its memory access latency, measured with the Intel’s Event Based Sampling (PEBS) [28]. The Roofline model [29], [30] analyzes compute performance and memory bandwidth. The application is classified as compute or memory bound based on the comparison with the performance roofs of the target hardware platform. The Top-down model [31] analyzes the application CPI stack. The application is categorized as memory bound if its significant CPI component is caused by the main memory accesses. The model also distinguishes between the memory latency and bandwidth stalls depending on the occupancy of the memory controller queues.

VIII. CONCLUSIONS

The **Memory stress (Mess) framework** offers a comprehensive and unified approach to memory system benchmarking, simulation, and application profiling. The **Mess benchmark** provides a detailed and holistic view of memory systems performance. It covers x86, ARM, Power, RISC-V, and NVIDIA’s PTX ISAs, and it is already deployed to characterize servers from Intel, AMD, IBM, Fujitsu, Amazon, and NVIDIA with DDR4, DDR5, HBM2, and HBM2E main memory. The **Mess analytical memory simulator** is integrated with ZSim, gem5, and OpenPiton Metro-MPI CPU simulators and it supports a wide range of memory systems, including the CXL memory expanders. It closely matches the actual memory systems performance and it shows an unprecedented low error of 0.4–6% when simulating widely-used memory benchmarks. Finally, the **Mess application profiling** is already integrated into a suite of production HPC performance analysis tools, adding a new layer of information related to application’s memory performance metrics. The Mess framework is publicly-released and ready to be used by the community for better understanding of the current and exploration of future memory systems.

IX. ACKNOWLEDGMENT

This work was funded by the Collaboration Agreement between Micron Technology, Inc. and BSC. The results have been partially funded by the Spanish Ministry of Science and Innovation MCIN/AEI/10.13039/501100011033 (contracts PID2019-107255GB-C21, PCI2022-132935, PCI2021-121958, PID2023-146511NB-I00 and CEX2021-001148-S), the Generalitat of Catalunya (contract 2021-SGR-00763, 2021-SGR-00807 and 2021-SGR-01264), European Union (Grant Agreement No. 955606), European Processor Initiative (EPI) with grant agreement No. 800928, 826647 and 101036168, and by the Barcelona Zettascale Lab (BZL) project with reference REGAGE22e00058408992 and co-financed by the Ministry for Digital Transformation and Public Services, within the framework of the Resilience and Recovery Fund – and the European

Union – NextGenerationEU. The work is also supported by the Arm-BSC Center of Excellence and the European HiPEAC Network of Excellence. A. Armejach is a Serra Hunter Fellow. We would also like to acknowledge valuable comments from the following individuals: Julian Pavon (BSC), Mohammad Bakhshalipour (CMU), Vicenc Beltran (BSC), Arash Yadegari (BSC), Nam Ho (FJZ) and Carlos Falquez (FJZ), Stepan Vanecek (TUM), John McCalpin (TACC).

APPENDIX

A. Mess benchmark implementation

In this appendix, we discuss Mess benchmark implementation. It includes implementation of pointer-chase and traffic generator. The scripts and manuals for the experimental setup, execution and measurements is included in the public Mess repository. The measurements post-processing removes the outliers, mitigates the noise and plots the results.

1) *Pointer-chase*: The pointer-chase contains a sequence of dependent back-to-back load instructions that access the main memory. Since the instructions are dependent, their execution is serialized, so the average memory access latency is calculated as the ratio between the total execution time and the number of instructions executed. This is guaranteed by the pointer-chase design. In the benchmark initialization, we allocate a contiguous section of memory and initialize it in such a way that a given array element contains the address of the next array element (memory location) that we want to access. This ensures the dependency between the consecutive load instructions. To force accesses to the main memory (and not on-chip caches), the pointer-chase traverses the whole array whose size exceeds the last-level cache. To avoid any cache-line spatial locality, each array element occupies the whole cache line (64 Byte). Finally, to diminish the impact of data prefetching and temporal locality during traversing, the pointer-chase traverses the array in a random pattern. The pointer-chase source code is detailed in Listing 1:

- Lines 0001–0004 initialize the benchmark: set the number of load instructions, initialize the registers, etc.
- Lines 0006–1007 are the core of the benchmark. They are a series of back-to-back load instructions for traversing the array. Each load instruction has data dependency on the previous one, so their execution is serialized.
- Line 1008 decrements the loop counter.
- Line 1009 checks the loop exit condition.

2) *Traffic generator*: We write the core of the memory traffic generator directly in assembly. This enables the development of more precise code (changes at the level of the assembly instruction) and prevents any system software optimizations (e.g. compiler optimizations). The benchmark generates different read and write memory traffic by executing a different mix of load and store instructions in the source code, each of them in a separate assembly file. In the current implementation, the benchmark can execute from 100% load

Listing 1: Pointer-chase source code. x86 assembly. In our study we also use the benchmark for the ARM, IBM Power, and RISC-V ISA.

Line	Source code	Explanation
0001	register struct line *next asm("rax");	struct line owns pointer to the next access
0002	register int i asm("ecx");	ecx is the loop counter
0003	i = 1000000;	initialization of the loop counter. C format.
0004	next = ptr->next	first memory access in C format
0005	start_loop:	beginning of the loop
0006	mov (%rax), %rax	load instruction (pointer-chase)
0007	mov (%rax), %rax	load instruction (pointer-chase)
...	...	...
1007	mov (%rax), %rax	load instruction (pointer-chase)
1008	dec %ecx	decrement loop counter
1009	jnz start_loop	if counter is not zero jump to start of the loop

to 100% store operations, with a step of 2%.⁷ The generated memory bandwidth is adjusted the issue rate of the memory operations. This is done by interleaving the load/store operation sequence with a configurable dummy loop of `nop` operations. To reduce the overhead of the loop management, loop counter decrement and jump operations, we employ a long loop of around 100 load/store operations. Listings 2, and 3 show the memory traffic generator implementation in the x86 ISA. The benchmark is also available for ARM, IBM Power, and RISC-V ISAs.

- Lines 001–008 initialize the benchmark:
 - Lines 001–002 loads the addresses of the arrays `a` and `c` into the registers.
 - Lines 003–004 initialize a register as the loop counter.
 - Lines 005–006 set the size of the array `a` and `c`.
 - Lines 007–008 set the iteration of the dummy `nop` function. This controls the rate of the memory operations, i.e. used memory bandwidth.
- The main loop of the kernel starts at line 009. The loop comprises a series of load and store instructions (Lines 010–015) followed by the call to the dummy `nop` function (Lines 016–017).
 - The `nop` function is shown in Listing 3. The number of `nop` operations is configurable: the larger the number of `nop` operations in each function call, the lower the rate of memory instruction, and therefore the lower used memory bandwidth. This configuration is managed by the value of `nopCount` parameter. When `nopCount` parameter is set to 0, `i=0` in Listing 3, the function is finalized without entering the `nop` loop. This causes only a negligible halt in the load and store sequence, leading to the maximum pressure to the memory system. As we increase the value of the `nopCount` parameter, we increase the number of the iterations in the dummy `nop` loop, and

⁷The ratio of loads and stores executed by the benchmark does not lead to the same ratio of the reads and writes in the memory traffic. This is because most of the state-of-the-art HPC servers deploy a write-allocate cache policy. With this policy each store operation targeting data in main memory is first read into the cache (memory read), then modified, and finally written to the main memory once the cache line is evicted. Since each store operation causes one memory read and one memory write, e.g. 100% store kernel causes 50%-read 50%-write memory traffic.

Listing 2: Memory traffic generator, x86 ISA. Different read and write memory traffic is generated by executing a different mix of load and store instructions in the source code, each of them in a separate assembly file. The generated memory bandwidth is adjusted the issue rate of the memory operations. This is done by interleaving the load/store operation sequence with a configurable dummy loop of `nop` operations.

Line	Source code	Explanation
001	register double *a asm("r10");	load array <code>a</code> address into the register <code>r10</code>
002	register double *c asm("r11");	load array <code>c</code> address into the register <code>r11</code>
003	register ssize_t i asm("rbx");	register <code>rbx</code> holds the loop counter
004	i = 0;	initialize the counter (<code>i</code>) by 0
005	register ssize_t n asm("r12");	<code>r12</code> holds the size of the arrays
006	n = *array_size;	load <code>r12</code> with size of the arrays
007	register int *p asm("r15");	register <code>r15</code> holds the <code>nopCount</code> value
008	p = nopCount;	load <code>r15</code> with value of <code>nopCount</code>
009	..L_50:	the beginning of the kernel loop
010	vmovupd (%r10,%rbx,8), %ymm0;	load instruction
011	vmovupd %ymm1, (%r11,%rbx,8);	store instruction
012	vmovupd 32(%r10,%rbx,8), %ymm0;	load instruction
013	vmovupd %ymm1, 32(%r11,%rbx,8);	store instruction
014	vmovupd 64(%r10,%rbx,8), %ymm0;	load instruction
015	vmovupd %ymm1, 64(%r11,%rbx,8);	store instruction
016	movq %r15, %rdi;	send number of <code>nopCount</code> as an input of <code>nop</code> function
017	call nop;	call <code>nop</code> function
...	...	...
...	...	...
138	vmovupd 1536(%r10,%rbx,8), %ymm0;	load instruction
139	vmovupd %ymm1, 1536(%r11,%rbx,8);	store instruction
140	vmovupd 1568(%r10,%rbx,8), %ymm0;	load instruction
141	vmovupd %ymm1, 1568(%r11,%rbx,8);	store instruction
142	movq %r15, %rdi;	send number of <code>nopCount</code> as an input of <code>nop</code> function
143	call nop;	call <code>nop</code> function
144	add \$200, %rbx;	increment the loop counter
145	cmp %r12, %rbx;	check the exit condition
146	jb ..L_50;	conditional jump to beginning of the kernel loop

Listing 3: Dummy `nop` function is used to generate a pause in the sequence of the load and store instructions in the Memory traffic generator (see Listing 2). The input parameter `nopCount` is the number of the dummy loop iterations. The larger the number of iterations, the lower the rate of memory instruction and the lower the generated memory bandwidth.

Line	Source code	Explanation
001	int nop(int *nopCount) {	definition of <code>nop</code> function
002	unsigned int i = *nopCount;	<code>i</code> is the loop counter; number of loops for <code>nop</code> operations
003	if (!i) {	if <code>i</code> is zero then return immediately
004	return 0;	
005	} else {	assembly section of <code>nop</code> function
006	asm(	it is written in extended assembly format
007	mov %0, %%ecx;	move the value of <code>%0</code> register into <code>ecx</code> register
008	the_loop%=:	the beginning of the loop
009	nop;	<code>nop</code> instruction
010	dec %%ecx	decrementing the counter
011	jnz the_loop%=:	conditional jump to beginning of the loop
012	: "r" (i) : ecx	put the value of <code>i</code> into register <code>%0</code> (extended assembly)
013	);	
014	return 0;	
015	}	

therefore reduce the rate of the memory operations issued by the memory traffic generator.

- The sequence of interleaved load and store instructions, and calls to the `nop` function is repeated until the line 143. This is the core of the benchmark. In the current implementation, each iteration of the main loop comprises 100 load and store instructions.
- Lines 144–146 finalize the loop iteration: increment the loop counter, check the exit condition, and conditionally jump to the beginning of the loop.

B. Case study: Remote-socket memory emulation of CXL memory expanders

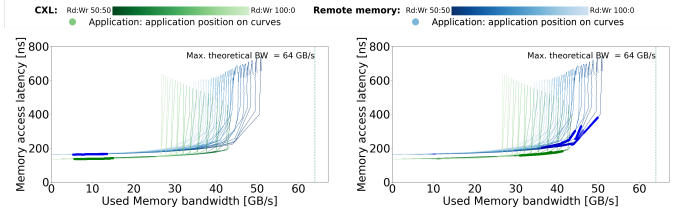
Recent industrial studies emulate CXL memory expansion in cloud and datacenter servers with conventional dual-socket systems: one socket is used as a host CPU and the other socket as a CPU-less memory expander [102], [103]. To evaluate this approach, we perform a detailed simulation of both systems. The CPU host is simulated with the ZSim configured to model the Intel 24-core Skylake processor (Section IV-B). We use Micron Technology’s bandwidth–latency curves to model the CXL memory expander, and measurements from the actual dual-socket server to model the remote-socket memory. In each system configuration, we simulate 500 billion instructions of the multiprogrammed SPEC CPU2006 workloads [104].

Figure 17 plots the CXL (green) and remote-socket (blue) memory curves, and the benchmark behavior. Each 20,000 simulated instructions we monitor the benchmark read and write memory bandwidth, and then plot each observation as a point on the corresponding bandwidth–latency curve. The figure shows two characteristic use-cases. The perlbench benchmark (Figure 17(a)) has a low bandwidth utilization. In this area remote-socket memory curves shows approximately 28 ns higher latency, which leads to somewhat lower performance w.r.t. target CXL system. We detect the opposite behavior for high-bandwidth benchmarks. The remote-socket curves have a higher bandwidth saturation area. This leads to higher achieved bandwidths and the overall performance of the bandwidth-intensive benchmarks, such as the lbm illustrated in Figure 17(b). The overall performance trend is illustrated in Figure 18. The figure plots the results for all SPEC CPU2006 benchmarks, sorted from the lowest to the highest bandwidth utilization. For low-bandwidth benchmarks, remote-socket emulation provides up to 12% lower performance w.r.t. to the target CXL system. Both system provide a similar performance for the benchmarks with medium bandwidth utilization between 30% and 50% of CXL max theoretical bandwidth. For the high-bandwidth benchmarks, the remote-socket memory provides 11%–22% higher performance.

To the best of our knowledge, this is the first study that enables a detailed simulation of CXL memory expanders based on the manufacturer’s SystemC model. The model is already integrated and publicly released with ZSim, gem5 and OpenPiton Metro-MPI simulators. We also provide the performance analysis and correction factors that can be useful for future studies who want to keep using remote-socket memory emulation, e.g. due to its much faster execution time.

C. Artifact Appendix

1) *Abstract:* This artifact comprises the implementation of the Mess benchmark for various platforms, including actual hardware platforms (e.g., Intel, IBM, NVIDIA), system simulators (ZSim, gem5, and OpenPiton), and memory simulators (DRAMsim3, Ramulator, Ramulator2). It also includes the Mess memory simulator integrated with ZSim, gem5, and OpenPiton simulators.



(a) The perlbench experiences higher remote-socket memory latency which leads to 5% lower performance. (b) The lbm reaches higher remote-socket memory bandwidth which leads to 11% higher performance.

Fig. 17: Remote-socket emulation of CXL memory expanders: Bandwidth–latency curves and implications

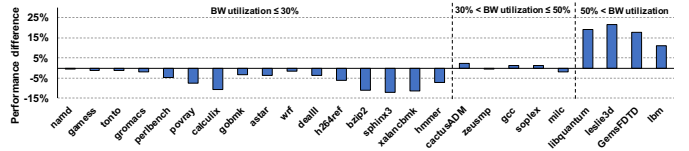


Fig. 18: Remote-socket memory emulation of CXL memory expanders: Application performance difference is correlated with its bandwidth utilization. SPEC2006 benchmarks, 24-core Intel Skylake CPU.

This artifact also contains all the scripts and guidelines necessary to reproduce the major figures presented in this paper. In addition to the scripts, it includes raw hardware measurements, processed measurements, and the final bandwidth–latency curves.

This study involves more than eight different hardware platforms with a diverse set of runtime environments, including various compilers, ISAs, and tools. Therefore, in this artifact, we mention the packages, tools, and applications without specifying the exact versions. In the Git repository, we provide detailed dependencies and version information for each experiment.

2) Artifact check-list (meta-information):

- **Program:** Pointer-chase and workload generator implemented in C, C++ with its kernel implemented in inline assembly (included in the benchmark). Mess simulator integrated with ZSim, gem5, and OpenPiton.
- **Compilation:** GCC, G++, ICX, MPI++, and Python 3.
- **Data set:** All the raw values measured from hardware counters and simulation traces as well as final curves for all the figures are included.
- **Run-time environment:** For Intel Cascade Lake curves, we use small server with single node. For Fujitsu A64FX, we use PJM batch processing support. For Graviton server, we use Amazon AWS. For the rest of the systems, we use production servers with Slurm Workload Manager environment.
- **Hardware:** Servers or Supercomputers with the following CPUs and GPUs: Intel Sandy Bridge, Skylake, Cascade Lake and Sapphire Rapids. AMD Zen 2, IBM Power9, Amazon Graviton 3, Fujitsu A64FX, NVIDIA Hopper H100.
- **Metrics:** For latency, we use nanoseconds and cycles. For bandwidth, we use GB/s.

- **Output:** We plot bandwidth–latency curves. Moreover, we print detailed data points in a .csv file.
- **Experiments:** Generate experiments using supplied scripts.
- **How much disk space required (approximately)?:** 10s of GB.
- **How much time is needed to prepare workflow (approximately)?:** For each experiment, approximately one hour.
- **How much time is needed to complete experiments (approximately)?:** To generate bandwidth–latency curves for actual hardware, ZSim, gem5, and OpenPiton, we need approximately 3-6 days, 1-2 weeks, 2-3 weeks, and 1-2 weeks, respectively.
- **Publicly available:** Yes.
- **Code licenses:** MIT License.
- **Data licenses:** MIT License.
- **Archived (provide DOI)?:** 10.5281/zenodo.13748673

3) *Description:* Figure 19 and 20 show where to replicate each major result presented in the paper. A detailed explanation of dependencies, system configurations, experimental setups, and result validation is available in the README.md file in each folder. To fit within the page limit of this artifact, this guideline introduces the general approach for replicating each result, along with an example to reproduce Figure 6.b. To replicate other figures, a similar approach should be followed (detailed guidelines are available in the Git repositories).

a) *How to access:* The Mess Benchmark artifact can be cloned from GitHub at <https://github.com/bsc-mem/Mess-benchmark.git>. The structure of the repository is depicted in Figure 19. The Mess simulator artifact can also be cloned from <https://github.com/bsc-mem/Mess-simulator>. The structure of the repository is depicted in Figure 20. Each folder in the repositories replicates one or more figures presented in the main manuscript (figures are indicated in blue text). This artifact can also be downloaded as a .zip file from <https://zenodo.org/records/13748674>.

b) *Hardware dependencies:* To run the Mess benchmark on actual hardware, access to a full node is required. The CPU/GPUs must support hardware counters to measure memory bandwidth (preferably uncore counters). For simulation experiments, a single core is sufficient. However, ZSim and OpenPiton can benefit from multicore or multinode parallelism.

c) *Software dependencies:* The benchmark and simulations run on Linux OS. To measure uncore counters, we primarily use the Linux perf tool, which is supported by all major Linux versions. In some cases, we also use Intel VTune and LIKWID.

d) *Data sets:* All the data sets are included in the repositories.

4) *Installation:* To install, first clone the repository. Then, navigate to the directory corresponding to the figure from the main manuscript that you want to replicate (see Figures 19 and 20). For Figure 6.b of the main manuscript:

```
git clone https://github.com/bsc-mem/Mess-benchmark.git
cd ./CPU/Simulators/Trace-driven/DRAMsim3
```

5) *Experiment workflow:* For each experiment (each folder in Figures 19 or 20), the workflow for running the Mess benchmark/simulation is provided in the runner.sh script. For Figure 6.b of the main manuscript (trace-driven DRAMsim3 simulation), the full workflow takes less than one day.

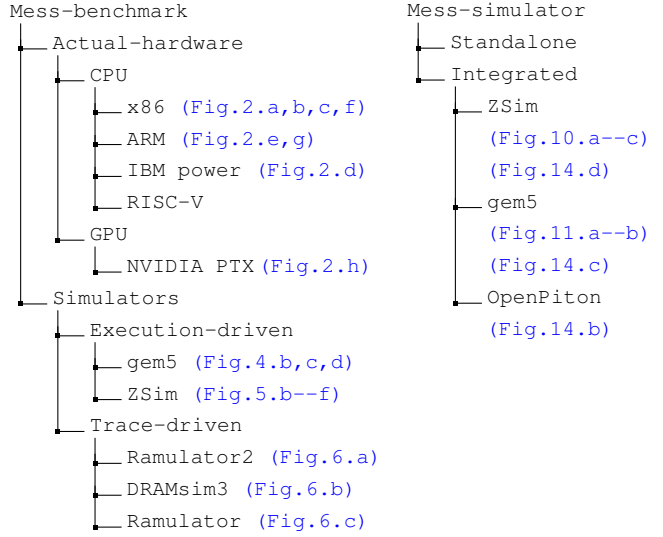


Fig. 19: Mess benchmark repo Fig. 20: Mess simulator repo

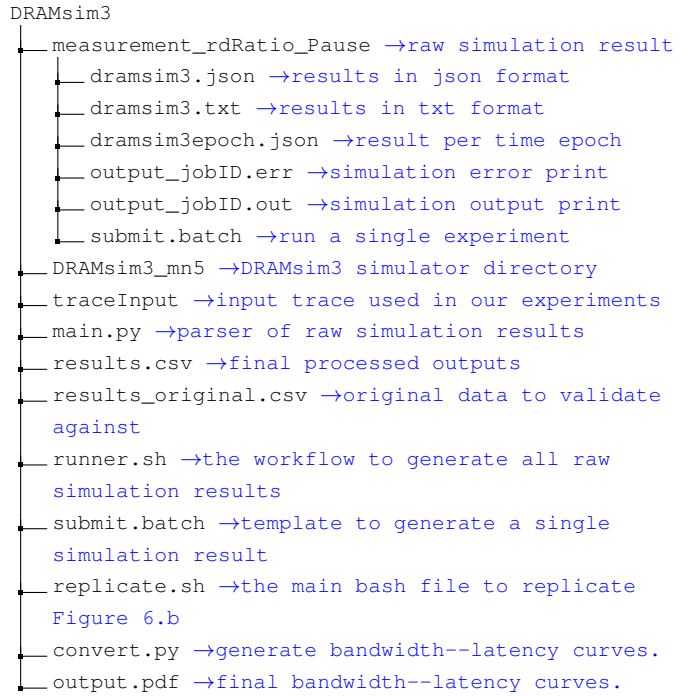


Fig. 21: Directory structure to replicate DRAMsim3 experiments.

6) *Evaluation and expected results:* To replicate each experiment, we run replicate.sh script. This script compiles necessary codes, executes the workflow (i.e., runner.sh), and processes the output raw data. The artifact also includes all the raw measurements and final processed .csv data. For the DRAMsim3 example, the replicate.sh script inside DRAMsim3 directory (Figure 21) executes the following commands:

```
# unzip trace files
cd traceInput
```

```

for file in *.zip; do
    unzip "$file"
done
cd ..

# compile DRAMsim3
cd DRAMsim3_mn5
mkdir build
cd build
cmake ..
make
cd ../..

# run the experiment
./runner.sh

#####
# Post-processing #
#####

# generate results.csv file
python3 main.py .

# generate output.pdf (bandwidth--latency curves)
python3 convert.py

```

The easiest way to validate the result is visually by examining the generated curves (e.g., output.pdf in our example). However, if one wants to evaluate the results in more detail, the results.csv file can be compared to results_original.csv; rows with the same rw_ratio and pause values should have a very close latency and bandwidth.

REFERENCES

- [1] W. A. Wulf and S. A. McKee. Hitting the memory wall: Implications of the obvious. In *SIGARCH*, pages 20–24. Comput. Archit. News, March 1995.
- [2] Richard Sites. It’s the memory, stupid! *Microprocessor Report*, pages 2–3, August 1996.
- [3] A. Saulsbury, F. Pong, and A. Nowatzky. Missing the memory wall: The case for processor/memory integration. In *Proceedings of the 23rd Annual International Symposium on Computer Architecture, ISCA*, pages pages 90–101, New York, 1996. ACM.
- [4] Intel. Intel Advisor. <https://www.intel.com/content/www/us/en/developer/tools/oneapi/advisor.html>, 2021. [Online; accessed 27-June-2023].
- [5] John D McCalpin. Memory bandwidth and machine balance in current high performance computers. *IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter*, 1995.
- [6] Wenqing Peng and Evgenij Belikov. Camp: a synthetic microbenchmark for assessing deep memory hierarchies. In *2022 IEEE/ACM International Workshop on Hierarchical Parallelism for Exascale Computing (HiPar)*, 2022.
- [7] j. Dongarra, M. Heroux, and P. Luszczek. “The HPCG Benchmark,” <http://www.hpcg-benchmark.org>, 2016.
- [8] LMBench. <http://lmbench.sourceforge.net>, 12 2005.
- [9] Google. Multichase. <https://github.com/google/multichase>, 2021.
- [10] R. S. Verdejo and P. Radojkovic. Microbenchmarks for Detailed Validation and Tuning of Hardware Simulators. In *International Conference on High Performance Computing Simulation (HPCS)*, 2017.
- [11] Intel Corporation. Intel memory latency checker v3.5. <https://software.intel.com/en-us/articles/intelr-memory-latency-checker>, 2023.
- [12] Mark Gottscho, Sriram Govindan, Bikash Sharma, Mohammed Shoaib, and Puneet Gupta. X-mem: A cross-platform and extensible memory characterization tool for the cloud. In *2016 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2016.
- [13] Trevor E. Carlson, Wim Heirman, Stijn Eyerman, Ibrahim Hur, and Lieven Eeckhout. An Evaluation of High-Level Mechanistic Core Models. *ACM Trans. Archit. Code Optim.*, 11(3), August 2014.
- [14] Trevor E. Carlson, Wim Heirman, and Lieven Eeckhout. Sniper: Exploring the level of abstraction for scalable and accurate parallel multi-core simulation. In *SC ’11: Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12, November 2011.
- [15] Daniel Sanchez and Christos Kozyrakis. ZSim: fast and accurate microarchitectural simulation of thousand-core systems. In *ISCA’ 13 Proceedings of the 40th Annual International Symposium on Computer Architecture*, pages 475–486, 2013.
- [16] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adrià Armejach, Nils Asmussen, Brad Beckmann, Srikant Bharadwaj, Gabe Black, Gedare Bloom, Bobby R. Bruce, Daniel Rodrigues Carvalho, Jeronimo Castrillon, Lizhong Chen, Nicolas Derumigny, Stephan Diestelhorst, Wendy Elsasser, Carlos Escuin, Marjan Fariborz, Amin Farmahini-Farahani, Pouya Fotouhi, Ryan Gambord, Jayneel Gandhi, Dibakar Gope, Thomas Grass, Anthony Gutierrez, Bagus Hanindhito, Andreas Hansson, Swapnil Haria, Austin Harris, Timothy Hayes, Adrian Herrera, Matthew Horsnell, Syed Ali Raza Jafri, Radhika Jagtap, Hanhwi Jang, Reiley Jeyapaul, Timothy M. Jones, Matthias Jung, Subash Kannoth, Hamidreza Khaleghzadeh, Yuetsu Kodama, Tushar Krishna, Tommaso Marinelli, Christian Menard, Andrea Mondelli, Miquel Moreto, Tiago Mück, Omar Naji, Krishnendra Nathella, Hoa Nguyen, Nikos Nikoleris, Lena E. Olson, Marc Orr, Binh Pham, Pablo Prieto, Trivikram Reddy, Alec Roelke, Mahyar Samani, Andreas Sandberg, Javier Setoain, Boris Shingarov, Matthew D. Sinclair, Tuan Ta, Rahul Thakur, Giacomo Travaglini, Michael Upton, Nilay Vaish, Ilias Vougioukas, William Wang, Zhengrong Wang, Norbert Wehn, Christian Weis, David A. Wood, Hongil Yoon, and Éder F. Zulian. The gem5 Simulator: Version 20.0+, 2020.
- [17] Nagendra Gulur, Mahesh Mehendale, Raman Manikantan, and Ramaswamy Govindarajan. Anatomy: An analytical model of memory system performance. In *The 2014 ACM International Conference on Measurement and Modeling of Computer Systems, SIGMETRICS ’14*, 2014.
- [18] Jason E. Miller, Harshad Kasture, George Kurian, Charles Gruenwald, Nathan Beckmann, Christopher Celio, Jonathan Eastepe, and Anant Agarwal. Graphite: A distributed parallel simulator for multicores. In *HPCA - 16 The Sixteenth International Symposium on High-Performance Computer Architecture*, pages 1–12, 2010.
- [19] Sadagopan Srinivasan Li Zhao Brinda Ganesh, Bruce Jacob, and Mike Espig Ravi Iyer. CMP Memory Modeling: How Much Does Accuracy Matter? In *Fifth Annual Workshop on Modeling, Benchmarking and Simulation*, 2009.
- [20] Niladrish Chatterjee, Rajeev Balasubramonian, Manjunath Shevgoor, Seth Pugsley, Aniruddha Udiipi, Ali Shafiee, Kshitij Sudan, Manu Awasthi, and Zeshan Chishti. Usimm: the utah simulated memory module. *University of Utah and Intel, Tech. Rep.*, 2012.
- [21] David Wang, Brinda Ganesh, Nuengwong Tuaycharoen, Katie Baynes, Aamer Jaleel, and Bruce Jacob. DRAMsim: A memory-system simulator, November 2005.
- [22] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob. DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM Simulator. *IEEE Computer Architecture Letters*, 19(2):106–109, 2020.
- [23] Yoongu Kim, Weikun Yang, and Onur Mutlu. Ramulator: A Fast and Extensible DRAM Simulator. In *IEEE Computer Architecture Letters*, volume 15, pages 45–49, 2016.
- [24] Haocong Luo, Yahya Can Tu, F Nisa Bostancı, Ataberk Olgun, A Giray Ya, and Onur Mutlu. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator. *IEEE Computer Architecture Letters*, pages 1–4, November 2023.
- [25] Lukas Steiner, Matthias Jung, Felipe S Prado, Kirill Bykov, and Norbert Wehn. DRAMSys4. 0: An open-source simulation framework for in-depth DRAM Analyses. volume 50, pages 217–242. Springer, 2022.
- [26] Jeffrey Dean, James E Hicks, Carl A Waldspurger, William E Weihl, and George Chrysos. ProfileMe: Hardware support for instruction-level profiling on out-of-order processors. In *Proceedings of 30th Annual International Symposium on Microarchitecture*, pages 292–302. IEEE, 1997.
- [27] Christian Helm and Kenjiro Taura. Perfmemplus: A tool for automatic discovery of memory performance problems. In *International Conference on High Performance Computing*, pages 209–226. Springer, 2019.

- [28] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer's Manual, Volume 3B: System Programming Guide, Part 2*, November 2009.
- [29] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [30] Aleksandar Ilic, Frederico Pratas, and Leonel Sousa. Cache-aware roofline model: Upgrading the loft. *IEEE Computer Architecture Letters*, 13(1):21–24, 2013.
- [31] Ahmad Yasin. A top-down method for performance analysis and counters architecture. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 35–44. IEEE, 2014.
- [32] Mess benchmark. <https://github.com/bsc-mem/Mess-benchmark>, 2024.
- [33] Guillem López-Paradís, Brian Li, Adrià Arnejach, Stefan Wallentowitz, Miquel Moretó, and Jonathan Balkind. Fast Behavioural RTL Simulation of 10B Transistor SoC Designs with Metro-Mpi. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–6. IEEE, 2023.
- [34] CXL Consortium. Compute express link (cxl), 2020.
- [35] Thomas F Wenisch, Roland E Wunderlich, Michael Ferdman, Anastassia Ailamaki, Babak Falsafi, and James C Hoe. SimFlex: Statistical Sampling of Computer System Simulation. *IEEE Micro*, 26(4):18–31, 2006.
- [36] Barcelona Supercomputing Center Performance Tools. BSC Tools, 2023, Dec.
- [37] Bruce Jacob. *The Memory System: You Can't Avoid It, You Can't Ignore It, You Can't Fake It*. Morgan and Claypool Publishers, 2009.
- [38] Christian Helm and Kenjiro Taura. Measurement of Main Memory Bandwidth and Memory Access Latency in Intel Processors. Technical report, 2019.
- [39] Jian Yang, Juno Kim, Morteza Hoseinzadeh, Joseph Izraelevitz, and Steve Swanson. An Empirical Guide to the Behavior and Use of Scalable Persistent Memory. In *18th USENIX Conference on File and Storage Technologies (FAST 20)*, pages 169–182, Santa Clara, CA, February 2020. USENIX Association.
- [40] Joseph Izraelevitz, Jian Yang, Lu Zhang, Juno Kim, Xiao Liu, Amir-saman Memaripour, Yun Joon Soh, Zixuan Wang, Yi Xu, Subramanya R. Dulloor, Jishen Zhao, and Steven Swanson. Basic Performance Measurements of the Intel Optane DC Persistent Memory Module, 2019.
- [41] Mess simulator. <https://github.com/bsc-mem/Mess-simulator>, 2024.
- [42] Mess extension of paraver. <https://github.com/bsc-performance-tools/wxparaver>, 2024.
- [43] Norman P. Jouppi. Cache write policies and performance. In *Proceedings of the 20th Annual International Symposium on Computer Architecture, ISCA '93*, page 191–201, New York, NY, USA, 1993. Association for Computing Machinery.
- [44] Linux. perf: Linux profiling with performance counters. https://perf.wiki.kernel.org/index.php/Main_Page, 2023.
- [45] Dan Terpstra, Heike Jagode, Haihang You, and Jack Dongarra. Collecting performance data with PAPI-C. In *Tools for High Performance Computing 2009: Proceedings of the 3rd International Workshop on Parallel Tools for High Performance Computing, September 2009, ZIH, Dresden*, pages 157–173. Springer, 2010.
- [46] Jan Treibig, Georg Hager, and Gerhard Wellein. LIKWID: A Lightweight Performance-Oriented Tool Suite for x86 Multicore Environments. In *39th International Conference on Parallel Processing Workshops*, pages 207–216, 2010.
- [47] CUDA CUPTI: Cuda profiling tools interface. <https://docs.nvidia.com/cupti>, 2024.
- [48] Rommel Sánchez Verdejo and Petar Radojkovic. Microbenchmarks for Detailed Validation and Tuning of Hardware Simulators. In *International Conference on High-Performance Computing & Simulation (HPCS)*, pages 881–883, 2017.
- [49] Rommel Sánchez Verdejo, Kazi Asifuzzaman, Milan Radulovic, Petar Radojkovic, Eduard Ayguadé, and Bruce Jacob. Main memory latency simulation: the missing link. In *Proceedings of the International Symposium on Memory Systems, MEMSYS '18*, page 107–116. Association for Computing Machinery, 2018.
- [50] D. W. Westcott and V. White. Instruction sampling instrumentation. US Patent #5,151,981, September 1992.
- [51] Paul Drongowski, Lei Yu, Frank Swehosky, Suravee Suthikulpanit, and Robert Richter. Incorporating Instruction-Based Sampling into AMD CodeAnalyst. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 119–120, 2010.
- [52] M. Srinivas, B. Sinharoy, R. J. Eickemeyer, R. Raghavan, S. Kunkel, T. Chen, W. Maron, D. Flemming, A. Blanchard, P. Seshadri, J. W. Kellington, A. Mericas, A. E. Petruski, V. R. Indukuru, and S. Reyes. IBM POWER7 performance modeling, verification, and evaluation. *IBM Journal of Research and Development*, pages 1–4, 2011.
- [53] Xinxin Mei and Xiaowen Chu. Dissecting GPU Memory Hierarchy Through Microbenchmarking. *IEEE Transactions on Parallel and Distributed Systems*, 28(1):72–86, 2017.
- [54] Intel. *Intel 64 and IA-32 Architectures Optimization Reference Manual*, November 2009.
- [55] Milan Radulovic, Rommel Sánchez Verdejo, Paul Carpenter, Petar Radojkovic, Bruce Jacob, and Eduard Ayguadé. PROFET: Modeling System Performance and Energy Without Simulating the CPU. *SIGMETRICS Perform. Eval. Rev.*, 47(1):71–72, December 2019.
- [56] Russell Clapp, Martin Dimitrov, Karthik Kumar, Vish Viswanathan, and Thomas Willhalm. Quantifying the Performance Impact of Memory Latency and Bandwidth for Big Data Workloads. In *Proceedings of the 2015 IEEE International Symposium on Workload Characterization, IISWC '15*, pages 213–224. IEEE Computer Society, 2015.
- [57] JEDEC STANDARD. *DDR4 SDRAM*. JEDEC Solid State Technology Association, 2017.
- [58] Tom Deakin, James Price, Matt Martineau, and Simon McIntosh-Smith. GPU-STREAM v2.0: Benchmarking the Achievable Memory Bandwidth of Many-Core Processors Across Diverse Parallel Programming Models. In *High Performance Computing*, pages 489–507. Springer International Publishing, 2016.
- [59] A. Kashyap. *High Performance Computing: Tuning guide for AMD EPYC 7002 Series Processor*, 2020.
- [60] Satish Kumar Sadasivam, Brian W. Thompto, Ron Kalla, and William J. Starke. IBM Power9 Processor Architecture. *IEEE Micro*, 37(2):40–51, 2017.
- [61] B. Wheeler. Graviton3 Debuts Neoverse V1. Technical report, Linley Group Microprocessor, 2022.
- [62] Arijit Biswas. Sapphire Rapids. In *IEEE Hot Chips 33 Symposium (HCS)*, pages 1–22, 2021.
- [63] Fujitsu. A64FX Microarchitecture Manual. Technical report, 2019.
- [64] Jack Choquette. NVIDIA Hopper H100 GPU: Scaling Performance. *IEEE Micro*, 43(3):9–17, 2023.
- [65] Joel Hestness, Stephen W. Keckler, and David A. Wood. A comparative analysis of microarchitecture effects on cpu and gpu memory system behavior. In *IEEE International Symposium on Workload Characterization (IISWC)*, pages 150–160, 2014.
- [66] Stijn Eyerman, Wim Heirman, and Ibrahim Hur. DRAM Bandwidth and Latency Stacks: Visualizing DRAM Bottlenecks. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 322–331, 2022.
- [67] Markus Velten, Robert Schöne, Thomas Ilsche, and Daniel Hackenberg. Memory Performance of AMD EPYC Rome and Intel Cascade Lake SP Server Processors. In *Proceedings of the 2022 ACM/SPEC on International Conference on Performance Engineering, ICPE '22*, 2022.
- [68] Jan Laukemann, Thomas Gruber, Georg Hager, Dossay Oryspayev, and Gerhard Wellein. CloverLeaf on Intel Multi-Core CPUs: A Case Study in Write-Allocate Evasion. *arXiv preprint arXiv:2311.04797*, 2023.
- [69] Irma Esmer Papazian. New 3rd gen intel® xeon® scalable processor (codename: Ice lake-sp). In *Hot Chips Symposium*, pages 1–22, 2020.
- [70] Zsim+drsim3 simulation infrastructure for process in memory. <https://github.com/bsc-mem/zsim/tree/master>, 2022.
- [71] Pouya Esmaili-Dokht, Miquel Guiot, Petar Radojkovic, Xavier Martorell, Eduard Ayguadé, Jesus Labarta, Jason Adlard, Paolo Amato, and Marco Sforzin. $O(n)$ Key-Value Sort With Active Compute Memory. *IEEE Transactions on Computers*, 73(05):1341–1356, may 2024.
- [72] Drsim3. <https://github.com/umd-memsys/DRAMsim3>, 2024.
- [73] Geraldo F Oliveira, Juan Gómez-Luna, Lois Orosa, Saugata Ghose, Nandita Vijaykumar, Ivan Fernandez, Mohammad Sadrosadati, and Onur Mutlu. DAMOV: A New Methodology and Benchmark Suite for Evaluating Data Movement Bottlenecks. *IEEE Access*, 2021.
- [74] JEDEC STANDARD. *DDR3 SDRAM*. JEDEC Solid State Technology Association, 2009.
- [75] Jonathan Balkind, Michael McKeown, Yaosheng Fu, Tri Nguyen, Yanqi Zhou, Alexey Lavrov, Mohammad Shahradd, Adi Fuchs, Samuel Payne, Xiaohua Liang, Matthew Matl, and David Wentzlaff. OpenPiton: An Open Source Manycore Research Framework. In *Proceedings of the*

- Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS '16, page 217–232, New York, NY, USA, 2016. Association for Computing Machinery.
- [76] Jonathan Balkind, Katie Lim, Michael Schaffner, Fei Gao, Grigory Chirkov, Ang Li, Alexey Lavrov, Tri M. Nguyen, Yaosheng Fu, Florian Zaruba, Kunal Gulati, Luca Benini, and David Wentzlaff. BYOC: A "Bring Your Own Core" Framework for Heterogeneous-ISA Research. ASPLOS '20, page 699–714, New York, NY, USA, 2020. Association for Computing Machinery.
 - [77] Neiel Leyva, Alireza Monemi, Noelia Oliete-Escuín, Guillem López-Paradís, Xabier Abancens, Jonathan Balkind, Enrique Vallejo, Miquel Moretó, and Lluc Alvarez. OpenPiton Optimizations Towards High Performance Manycores. In *Proceedings of the 16th International Workshop on Network on Chip Architectures*, NoCArc '23, page 27–33, New York, NY, USA, 2023. Association for Computing Machinery.
 - [78] Jonathan Balkind, Katie Lim, Fei Gao, Jinzheng Tu, David Wentzlaff, Michael Schaffner, Florian Zaruba, and Luca Benini. OpenPiton+ Ariane: The first open-source, SMP Linux-booting RISC-V system scaling from one to many cores. In *Workshop on Computer Architecture Research with RISC-V (CARRV)*, pages 1–6, 2019.
 - [79] Wilson Snyder. Verilator and systemperl. In *North American SystemC Users' Group, Design Automation Conference*, 2004.
 - [80] Shang Li, Rommel Sánchez Verdejo, Petar Radojković, and Bruce Jacob. Rethinking Cycle Accurate DRAM Simulation. In *Proceedings of the International Symposium on Memory Systems*, MEMSYS '19, page 184–191, 2019.
 - [81] Stijn Eyerman, Wim Heirman, and Ibrahim Hur. Modeling dram timing in parallel simulators with immediate-response memory model. *IEEE Computer Architecture Letters*, 20(2):90–93, July 2021.
 - [82] Piotr R Luszczek, David H Bailey, Jack J Dongarra, Jeremy Kepner, Robert F Lucas, Rolf Rabenseifner, and Daisuke Takahashi. The HPC Challenge (HPCC) benchmark suite. In *Proceedings of the 2006 ACM/IEEE conference on Supercomputing*, volume 213, page 1, 2006.
 - [83] G Franklin, J.D. Powell, and Abbas Emami-Naeini. *Feedback Control Of Dynamic Systems*. 1994.
 - [84] Graham C. Goodwin, Stefan F. Graebe, and Mario E. Salgado. *Control System Design*. 2000.
 - [85] Ayaz Akram and Lina Sawalha. A Survey of Computer Architecture Simulation Techniques and Tools. *IEEE Access*, 7:78120–78145, 2019.
 - [86] Anastasiia Butko, Rafael Garibotti, Luciano Ost, and Gilles Sassatelli. Accuracy evaluation of GEM5 simulator system. In *7th International Workshop on Reconfigurable and Communication-Centric Systems-on-Chip (ReCoSoC)*, pages 1–7, 2012.
 - [87] Anthony Gutierrez, Joseph Pusdesris, Ronald G. Dreslinski, Trevor Mudge, Chander Sudanthi, Christopher D. Emmons, Mitchell Hayenga, and Nigel Paver. Sources of error in full-system simulation. In *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 13–22, 2014.
 - [88] A. Akram and L. Sawalha. x86 computer architecture simulators: A comparative study. In *IEEE 34th International Conference on Computer Design (ICCD)*, pages 638–645, 2016.
 - [89] Frank Ghenassia. *Transaction level modeling with SystemC*. Springer, 2005.
 - [90] Debendra Das Sharma. Compute express link (cxl): Enabling heterogeneous data-centric computing with heterogeneous memory hierarchy. *IEEE Micro*, 43(2):99–109, 2023.
 - [91] Barcelona Supercomputing Center Performance Tools. Paraver data browser, 2023, December.
 - [92] Vincent Pillet, Jes Us Labarta, Toni Cortes, Sergi Girona, and Jesús Labarta. PARAVER : A Tool to Visualizeand Analyze Parallel. 1995.
 - [93] Barcelona Supercomputing Center Performance Tools. Paraver – Tracefile description. Technical report, 2023, December.
 - [94] Barcelona Supercomputing Center Performance Tools. Extrae tracing framework, 2023, December.
 - [95] Michael Allen Heroux, Jack Dongarra, and Piotr Luszczek. HPCG benchmark technical specification. Technical report, Sandia National Lab.(SNL-NM), Albuquerque, NM (United States), 2013.
 - [96] Intel. Intel Xeon CPU Max Series Configuration and Tuning Guide. <https://www.intel.com/content/www/us/en/content-details/769060/intel-xeon-cpu-max-series-configuration-and-tuning-guide.html>, 2023.
 - [97] John D McCalpin. Bandwidth Limits in the Intel Xeon Max (Sapphire Rapids with HBM) Processors. In *International Conference on High Performance Computing*, pages 403–413. Springer, 2023.
 - [98] John D McCalpin. STREAM: Sustainable Memory Bandwidth in High Performance Computers. <https://www.cs.virginia.edu/stream/>, 2022.
 - [99] John D McCalpin. The STREAM 2 Benchmark. <https://www.cs.virginia.edu/stream/stream2/>, 2022.
 - [100] Alif Ahmed and Kevin Skadron. Hopscotch: a micro-benchmark suite for memory performance evaluation. In *Proceedings of the International Symposium on Memory Systems*, MEMSYS '19, page 167–172, New York, NY, USA, 2019. Association for Computing Machinery.
 - [101] PROFET. <https://github.com/bsc-mem/PROFET>, 2024.
 - [102] Hasan Al Maruf, Hao Wang, Abhishek Dhanotia, Johannes Weiner, Niket Agarwal, Pallab Bhattacharya, Chris Petersen, Mosharaf Chowdhury, Shobhit Kanaujia, and Prakash Chauhan. Tpp: Transparent page placement for cxl-enabled tiered-memory. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS 2023, 2023.
 - [103] Huaicheng Li, Daniel S. Berger, Lisa Hsu, Daniel Ernst, Pantea Zardoshti, Stanko Novakovic, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, Mark D. Hill, Marcus Fontoura, and Ricardo Bianchini. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ASPLOS 2023, 2023.
 - [104] John L. Henning. SPEC CPU2006 benchmark descriptions. In *ACM SIGARCH Computer Architecture News*, volume 34, pages Pages 1 – 17, September 2006.